OVER | SECURITY INTELLIGENCE

Basic Pentest Mobile

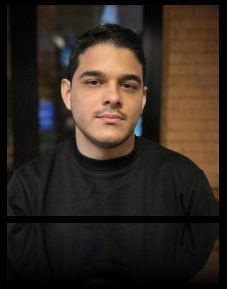
iOS



Instrutores

$ whoami

rodolfo.mariano



- Red Team Tech Lead na Compass UOL
- Instrutor Especialista na Over Security Intelligence
- Graduação em Técnologo de Análise e Desenvolvimento de Sistemas - UNIPÊ
- Pós-Graduado em Segurança Ofensiva e Inteligência Cibernética - VINCIT
- Extensão em Ciberguerra e Terrorismo - Charles Sturt University
- Certificações: OSCP, CEH, eMAPT, eWPTX, CRTP, eJPT, DCPT, SYCP, CWHI

$ whoami

caio.robaldo



- Red Team Tech Lead na Global Hitss/Claro Empresas
- Instrutor Especialista na Over Security Intelligence
- Graduação em Engenharia da Computação - Uniderp
- Pós-graduado em Computação Forense & Perícia Digital IPOG
- Pós-graduado em Direito Digital e Compliance
- Bacharel em Direito
- Pós-Graduado em Cyber Security - Ethical Hacker
- Certificações: CompTIA Pentest+, CMPEN-iOS

Pré-Requisitos

{01} Bases

Redes
Sistemas Operacionais
Programação

{02} Conhecimentos Básicos em Pentest



{03} iPhone

iPhone 6s e 7: até iOS 15.8.5 (A9,A10)
iPhone 8: até iOS 16.7.12 9 (A11)
iPhone X,XS,XR,11,SE e 12: até iOS 16.5.1 (A12,A13,A14)
iPhone 13, SE e 14: até iOS 16.0 (A15)

O que você vai aprender durante o workshop?

- Conceitos fundamentais de arquitetura e segurança do iOS
- Como realizar Jailbreaks e ajuste de ambiente para Pentests em app iOS
- Proteções como Jailbreak Detection, Anti-Hooking, TLS Pinning, mTLS
- Visão de atacante
- Dicas de carreira em Pentest Mobile

OVER SECURITY INTELLIGENCE

./intro

Pentest app iOS



Testes em Android são (relativamente) fáceis

- Open Source
- *Bootloader* Desbloqueável
- Modificação Fácil das Bibliotecas de Sistema
- *Sideloading* e Reempacotamento Simples: Instalar e modificar APKs fora da Play Store é rápido e direto.
- Diversidade de Dispositivos Físicos e Virtuais: Ampla gama de dispositivos e emuladores cobrindo diferentes versões do Android.

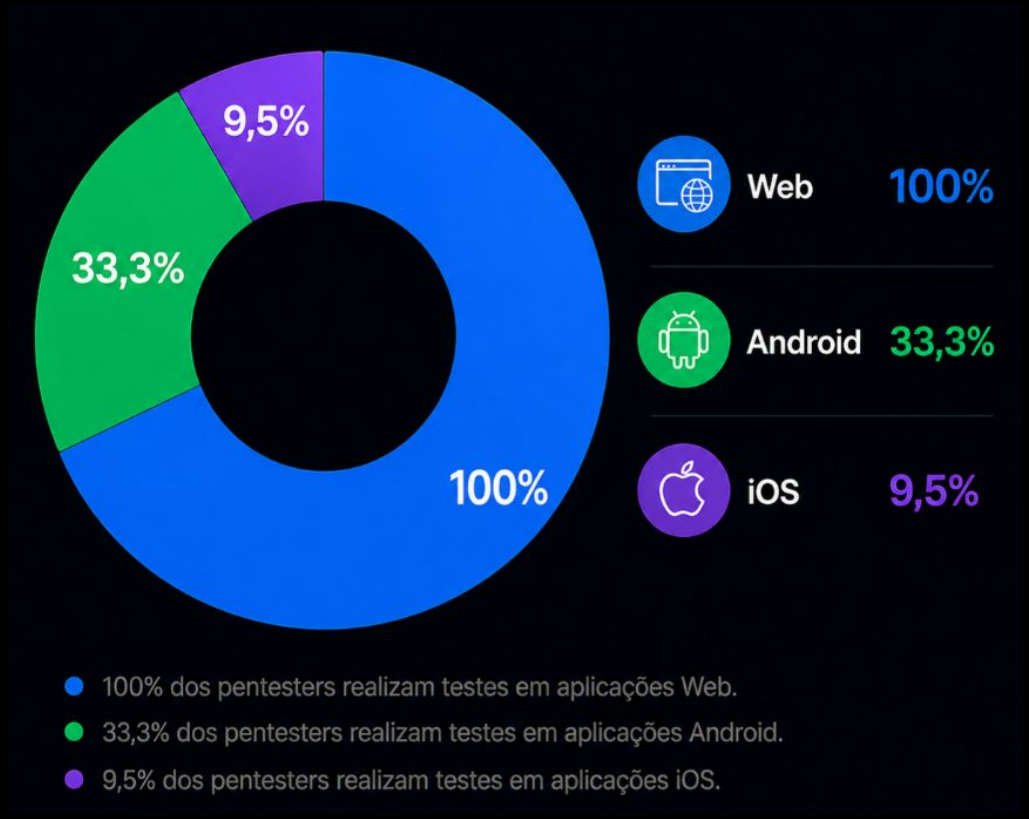


Testes em iOS

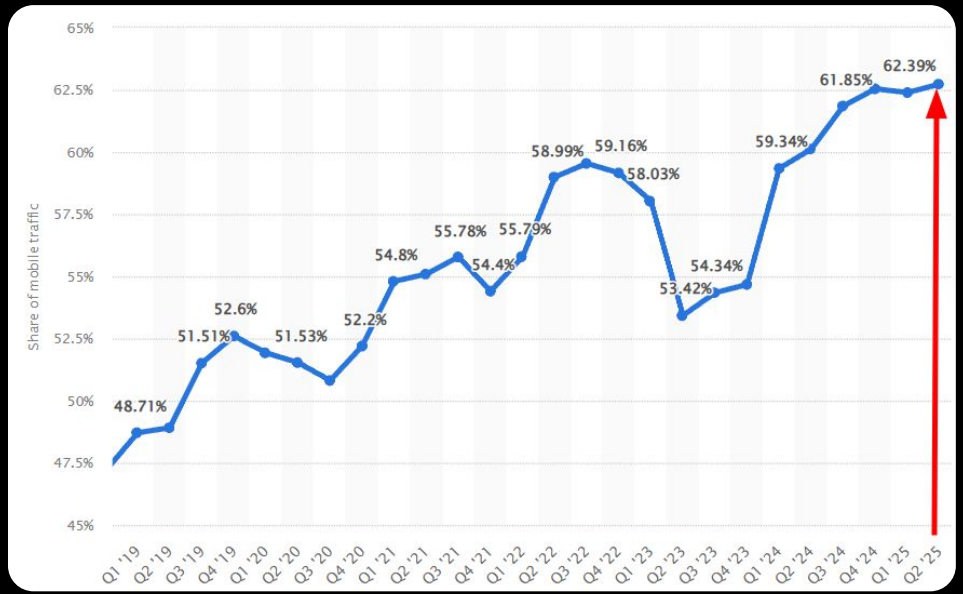


- Código-Fonte fechado: iOS é proprietário, sem acesso ao código-fonte.
- Sem Jailbreak em iPhones recentes: Os dispositivos mais novos têm poucas ou nenhuma opção de jailbreak.
- Preso à versão fixa do iOS: Normalmente não é possível escolher ou voltar versões do sistema.
- Dispositivos não modificáveis: Sem acesso root e sem modificações no sistema.
- Sem emulador realista: O simulador oficial da Apple (Xcode) é limitado e não replica hardware real.
- Acesso restrito ao SO: O sandbox do iOS limita o que podemos acessar e testar.
- Assinatura de código e binários criptografados: Todos os apps precisam ser assinados, e os binários são criptografados, dificultando análise dinâmica e engenharia reversa.

Pentesting iOS apps



BREAKING NEWS Os celulares foram responsáveis por mais de 62% de todo o tráfego web em 2025.



Fonte: https://www.statista.com/

Mais estatísticas:

- Mais de 8,5 bilhões de usuários de celulares em todo o mundo
- Cada pessoa passa em média quase 4 horas por dia no celular
- Mais de 88% desse tempo é gasto em aplicativos móveis

Tendências futuras e desafios

- Integração com IoT
- Impacto da IA
- Aumento das regulamentações
- BYOD (*bring your device*) e modelos de segurança *Zero Trust*
- Mudanças constantes nos *softwares* utilizados
- *Hardening* cada vez maior das versões do iOS
- Virtualização dos *devices*

FASE PRÉ-TESTE

1. Definir a abordagem conforme:
- Expectativas do cliente
- Requisitos regulatórios (LGPD, GDPR, etc.)

2. Possíveis restrições:
- Apps difíceis de testar
- Necessidade de versões de teste ou ambientes específicos

3. Conhecer os dados:
- Identificar dados sensíveis na visão do cliente
- Dados de autenticação (credenciais, infos de usuário)
- PII (Informações Pessoais Identificáveis)
- Dados protegidos por lei ou regulamentações

4. Localizar os dados:
- Onde estão armazenados?
- Como são armazenados?
- Para onde são enviados?
- Como são enviados?

5. Identificar itens relacionados à segurança:
- Hashing
- Criptografia
- Encoding
- Armazenamento de tokens (API, sessão, etc.)
- Geradores de números aleatórios

RECON

- Propósito do app
- Quem é o desenvolvedor
- Setor da indústria e stakeholders
- Como o app funciona de forma geral

APP MAPPING

- Arquitetura do App:
  - Como o app funciona internamente (ex.: gerenciamento de sessões)
  - Como o app se comunica (protocolos, APIs, etc.)
  - Com quem o app se comunica (servidores, serviços externos)
- Threat Modeling (Modelagem de Ameaças):
  - Identificar possíveis vetores de ataque antes de começar os testes

EXPLORAÇÃO

- Nem todas as vulnerabilidades encontradas serão relevantes ou exploráveis.
- Avaliar cada falha considerando:
  - Potencial de dano
  - Facilidade de descoberta
  - Impacto sobre dados ou usuários

RELATÓRIO

- Sua qualidade como pentester será julgada pelo seu relatório:
  - Você é tão bom quanto o seu relatório
- Um bom relatório deve conter:
  - Resumo Executivo
  - Definição do Escopo
  - Metodologia utilizada
  - Vulnerabilidades encontradas (*Findings*)
  - Recomendações

“Você é tão bom quanto o seu relatório”

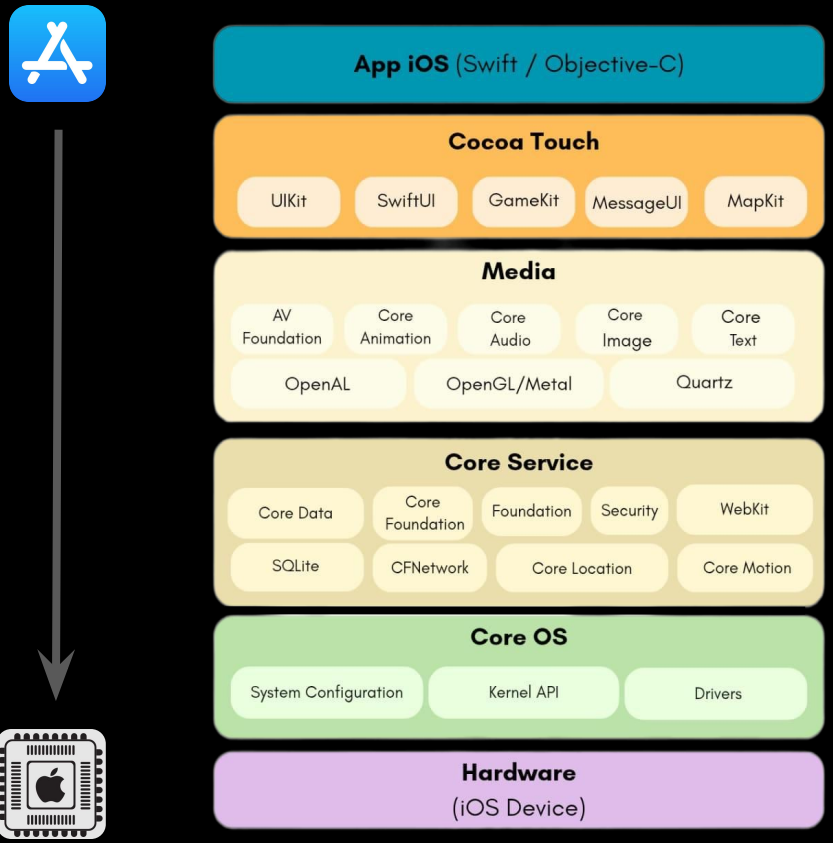
OVER SECURITY INTELLIGENCE

Arquitetura

iOS



Arquitetura iOS - Camadas



Arquitetura iOS - App iOS

App iOS (Swift / Objective-C)



App Web

Sites responsivos executados dentro de webviews, atuando como janelas específicas dentro de um aplicativo. São limitados para a nível do browser (ou sandbox).



App Híbrido

Aplicativos que utilizam webviews e permitem certas interações com funcionalidades nativas do sistema. Executam como se fossem um app nativo porém utilizam web browser.



App Nativo

São aplicativos desenvolvidos com a linguagem específica da plataforma, permitindo interação direta com o sistema operacional e seus recursos. Os custos são mais elevados no desenvolvimento e manutenção.

OVER SECURITY INTELLIGENCE

Recursos de Segurança iOS



Recursos de Segurança iOS

Os recursos de segurança do iOS detalhados no guia Apple Platform Security são projetados para proteger dados, rede e sistema, garantindo a integridade e o funcionamento seguro do dispositivo. Eles incluem criptografia de hardware, autenticação biométrica (Face ID e Touch ID), inicialização segura, atualizações protegidas, sandboxing de aplicativos e mecanismos avançados de segurança de rede e sistema.

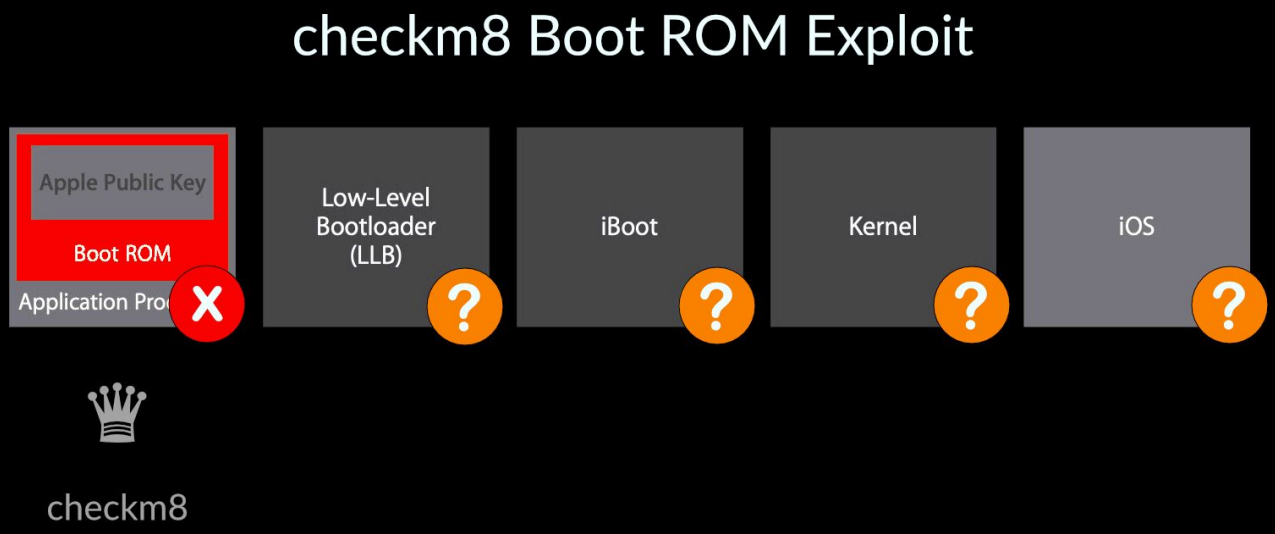


Secure Boot: Segurança de Hardware e Sistema

A **Secure Boot Chain** garante que apenas **código confiável seja executado desde a inicialização** do dispositivo. O processo começa no Boot ROM, código imutável gravado no chip que atua como Hardware Root of Trust. A partir dele, cada componente do sistema, como boot loader e kernel, é validado criptograficamente antes de ser executado, assegurando integridade e autenticidade. Essa cadeia de confiança combina segurança de hardware e software para impedir modificações maliciosas e dificultar ataques como jailbreak.



- CVE-2019-8900: É um exploit baseado em uma vulnerabilidade no Boot ROM de dispositivos Apple, requer acesso físico, e não é persistente após reinicialização.



Secure Enclave: Segurança de Hardware

- A **Secure Enclave** é um **subsistema seguro integrado ao SoC** (**chip**) da Apple, **isolado do processador principal** para proteger dados sensíveis mesmo se o kernel do processador de aplicativos for comprometido. Ele usa uma ROM de inicialização, criptografia AES e memória protegida para garantir a segurança. Embora não tenha armazenamento próprio, armazena dados em uma área separada do armazenamento principal do sistema.

- Para aplicativos iOS, essa arquitetura proporciona uma camada de segurança fundamental, isolando e protegendo informações críticas. A **Secure Enclave** assegura que operações de **autenticação biométrica e manipulação de dados sensíveis** sejam realizadas de forma segura, sem expor tais informações a outros componentes do sistema.

Secure Enclave - Suporte da Apple (BR)

Sandboxing: Segurança de Aplicativo

O sistema operacional iOS adota o modelo de App Sandbox como uma das principais medidas de segurança para proteger usuários e seus dados. Cada aplicativo é executado em um ambiente isolado (sandbox), o que impede interações diretas com outros apps ou com partes sensíveis do sistema operacional. Esse isolamento reduz significativamente a superfície de ataque e limita o impacto de eventuais falhas de segurança.

Code Signing e Criptografia FairPlay DRM

- O mecanismo de **Code Signing** (assinatura de código) é um dos pilares da segurança no iOS. Todos os aplicativos precisam ser assinados com um certificado válido emitido pela Apple, o que “garante” que apenas desenvolvedores autorizados possam distribuir software executável em dispositivos iOS. Isso impede a instalação e execução de código não verificado ou potencialmente malicioso.

- Através do **FairPlay DRM** a Apple também protege os binários distribuídos pela **App Store**, mantendo partes do executável **criptografadas** até o momento da execução, dificultando engenharia reversa, extração do app e redistribuição não autorizada.

Controles de Privacidade: Permissions and Entitlements

- No iOS, **entitlements** são direitos especiais solicitados pelo app para acessar recursos protegidos do sistema, como iCloud, App Groups ou Keychain. Eles são definidos no Xcode (aba *Signing & Capabilities*), salvos no arquivo .entitlements, e verificados pelo sistema no momento da instalação. Sem esses entitlements, o app não consegue usar certos recursos mesmo que o código esteja presente.

- Já as **permissions** são solicitações feitas ao usuário em tempo de uso, como acesso à câmera ou localização. Elas são declaradas no Info.plist e exigem uma justificativa visível ao usuário. O iOS permite que cada usuário gerencie essas permissões individualmente nas configurações do sistema.

- Além disso, o sistema implementa recursos como o **App Tracking Transparency (ATT)**, que exige consentimento do usuário antes de rastreamento entre apps e sites, reforçando a privacidade e o controle sobre os dados.

Network Security: App Transport Security

Por padrão, o App Transport Security (ATS) exige que os aplicativos iOS e macOS utilizem HTTPS com configurações de segurança fortes, o que inclui:

- Bloqueio de conexões HTTP (não seguras).
- Uso de TLS (geralmente versão 1.2 ou superior).
- Certificados válidos e corretamente configurados.

```
<key>NSAppTransportSecurity</key>
<dict>
    <key>NSAllowsArbitraryLoads</key>
    <true/>
</dict>
```

Insecure Local Networking: Essa é uma configuração no arquivo **Info.plist** que permite exceções às regras de segurança do ATS para redes internas (como servidores locais). desativando as restrições ATS para permitir conexões HTTP não seguras em redes locais (como servidores internos).

ATS AllowsArbitraryLoads: Configuração que, quando ativada no **Info.plist**, desabilita completamente as restrições do ATS. Isso permite que o aplicativo realize qualquer conexão, seja em redes locais ou públicas, usando HTTP ou HTTPS, sem seguir as verificações de segurança do ATS, como a exigência de TLS em uma versão mínima específica.

Mitigações Gerais de Exploits em iOS

Sistemas modernos adotam uma série de proteções para dificultar a exploração de vulnerabilidades por atacantes.

Essas técnicas, conhecidas como **exploit mitigations**, têm como objetivo tornar a exploração por atacantes significativamente mais difícil e muitas vezes inviável, forçando os atacantes a encontrar falhas mais complexas ou a depender de cadeias múltiplas de vulnerabilidades, protegendo assim a integridade e confidencialidade dos dados do usuário, mesmo diante de falhas no software.

Como por exemplo: ASLR, DEP/NX, PAC, Stack Canaries e CFI, mecanismos que dificultam exploração de memória, execução de código arbitrário e hijacking de fluxo de execução.



Analisando um .ipa

ESTRUTURA DE UM IPA



- **Payload/**
  - Diretório principal onde o pacote do aplicativo é armazenado
- **<app_name>.app/**
  - Diretório do pacote do aplicativo com todos os arquivos
- **Info.plist**
  - Arquivo de propriedades (nome, versão, permissões, etc...)
- **<app_name>**
  - Arquivo binário executável do aplicativo
- **_CodeSignature/CodeResources**
  - Arquivo de assinatura de código do aplicativo
- **embedded.mobileprovision**
  - Perfil de provisionamento com permissões e dispositivos autorizados para executar o aplicativo
- **.lproj**
  - Recursos de localização para múltiplos idiomas
- **Plugins/**
  - Extensões do app, como widgets ou notificações
- **Frameworks/**
  - Bibliotecas externas embarcadas que o aplicativo utiliza
- **.PkgInfo**
  - Arquivo pequeno com metadados de empacotamento
- **Assets.car**
  - Arquivo binário que contém recursos gráficos

OVER SECURITY INTELLIGENCE

Data Storage



Data Storage

- O **armazenamento de dados no iOS** pode ser realizado de várias formas, dependendo do tipo de dados, da sensibilidade, da persistência desejada, e do uso pretendido (local ou remoto). Aqui está um panorama dos principais mecanismos de armazenamento no iOS:
    - UserDefaults (NSUserDefaults) → preferências/configurações simples
    - SQLite → banco relacional local
    - PList Files → armazenamento simples baseado em arquivos
    - Keychain → armazenamento seguro de credenciais/tokens
    - Cache (NSURLCache, NSCache) → armazenamento temporário



- **/private/var/mobile/Containers/Data/Application/<UUID>/** (Dados/arquivos gravados durante a execução do aplicativo)
- **/private/var/containers/Bundle/Application/<UUID>/** (Código executável, recursos estáticos como imagens, storyboards, arquivos de configuração que foram embalados na build do app)

Keychain

Keychain é utilizado para armazenar dados sensíveis, como senhas, tokens, certificados e chaves criptográficas de forma nativa, fornecendo armazenamento isolado, criptografado e protegido pelo sistema.

Embora o Keychain seja seguro por padrão, ele pode ser comprometido através de dump via hooking com **Frida** ou **Objection**. Para mitigar esses riscos, recomenda-se:

- **Usar o atributo kSecAttrAccessibleWhenUnlockedThisDeviceOnly** para impedir a exportação dos dados para outros dispositivos; ativar entitlements corretos (**como Keychain Access Groups**);
- **Implementar Anti-Hook, Anti-Jailbreak, verificação de integridade e, quando possível, usar autenticação adicional (ex: Face ID / Touch ID via kSecAccessControl)** com suporte ao Secure Enclave, que armazena chaves criptográficas de forma isolada e resistente a engenharia reversa.
- Além disso, **evitar deixar chaves expostas em memória após o uso**, e sempre validar o uso legítimo antes de liberar dados sensíveis.

OVER SECURITY INTELLIGENCE

Utilitários

iOS



UDID

O UDID é um identificador único atrelado a cada dispositivo Apple, criado originalmente para permitir que desenvolvedores associem um dispositivo específico ao processo de distribuição ad hoc de aplicativos. Ele é essencial na assinatura e instalação de arquivos .ipa fora da App Store, já que o provisioning profile precisa conter os UDIDs autorizados. O UDID é utilizado também para:

Depuração e testes internos: Ferramentas como Xcode e TestFlight usam o UDID para identificar dispositivos durante testes controlados.

Controle de licenciamento: Softwares empresariais podem restringir o uso a dispositivos específicos por meio do UDID.

Gerenciamento de dispositivos (MDM): Soluções de gerenciamento corporativo usam o UDID para rastrear e aplicar políticas de segurança em dispositivos gerenciados.

➡ Instale o 3utools e veja na prática seu UDID

- https://3utools.org/3utools-download/

Simuladores e Emuladores

Simuladores

- Ferramentas que replicam o ambiente do iOS no Mac, como o Simulador do **Xcode**.
- Executam o código recompilado para a arquitetura do Mac, que pode ser Intel (x86_64) ou Apple Silicon (ARM64).
- Usados principalmente para desenvolvimento e testes funcionais rápidos.
- Limitados para testar funcionalidades dependentes de hardware real (GPS, sensores, câmera).

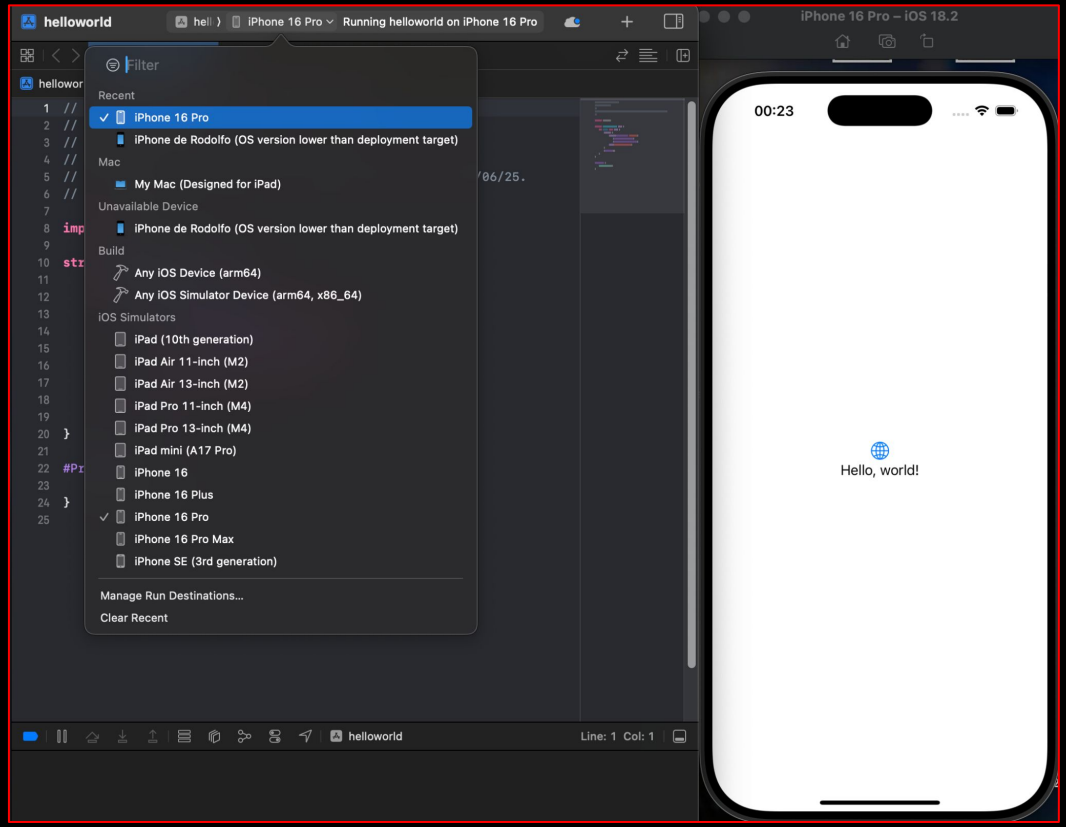


Emuladores

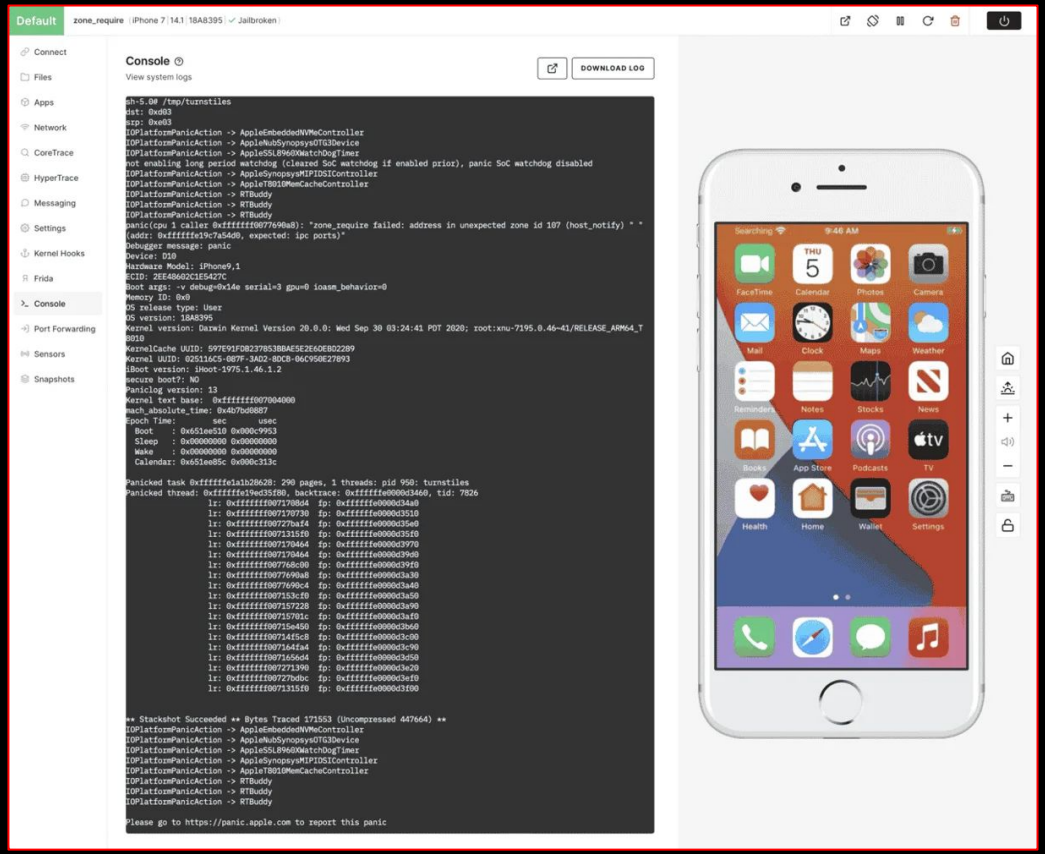
- Replicam o hardware e o sistema operacional real do dispositivo alvo. Exemplo: **Corellium**, que cria instâncias virtuais de iOS em arquitetura ARM.
- Permitem testes mais realistas, inclusive de baixo nível e segurança.
- Usados em pesquisa de vulnerabilidades, pentests e análise forense.



Simulador - Xcode



Emulador - Corellium



Transferindo dados entre o dispositivo non-jailbroken e o PC

Ferramentas como **iTunes, 3uTools** e **iMazing** permitem transferir arquivos, fazer backups e gerenciar dados do dispositivo.

Limitações importantes em dispositivos sem jailbreak:

- Sem jailbreak, o acesso é restrito ao sandbox do app e áreas de usuário.
- Não é possível acessar diretórios do sistema, raiz (root) ou arquivos protegidos do iOS.
- Isso garante segurança e integridade do sistema, mas limita análise e manipulação avançada de dados.

Transferindo dados entre o dispositivo jailbroken e o PC

Possibilidades em dispositivos com jailbreak:

- Com jailbreak, é possível acessar o **sistema de arquivos completo**, incluindo **diretórios raiz (root)**, arquivos de sistema e dados de outros apps.
- Ferramentas como **SSH/SCP** permitem conexão remota via terminal para extração ou modificação de arquivos.
- **Filza** oferece um gerenciador de arquivos com acesso total direto no dispositivo.
- **3uTools** e **iMazing** ampliam suas capacidades, podendo explorar mais diretórios além do sandbox tradicional.

OWASP MAS

MASVS

Mobile Application Security Verification Standard

Sven Schleier
Bernhard Mueller
Jeroen Beckers
Carlos Holguera
Jeroen Willemsen

OWASP

MASWE

Mobile Application Security Weakness Enumeration



Sven Schleier
Carlos Holguera
Jeroen Beckers

OWASP

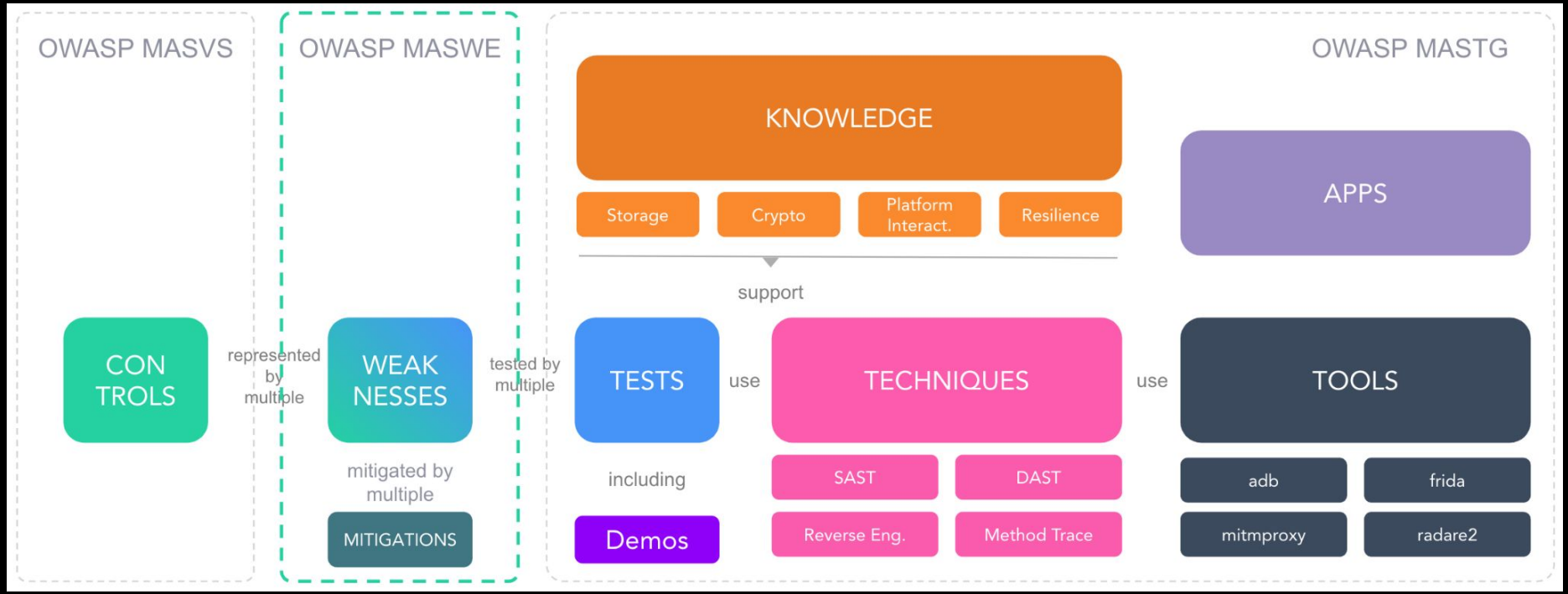
MASTG

Mobile Application Security Testing Guide

Sven Schleier
Bernhard Mueller
Carlos Holguera
Jeroen Willemsen

OWASP

OWASP MAS - Overview V2



OVER SECURITY INTELLIGENCE



JAILBREAK



Introdução

Jailbreak

Jailbreak é o processo que **remove as restrições da Apple no iOS**, permitindo a personalização do sistema, a instalação de apps fora da App Store e a adição de funcionalidades extras por meio de tweaks (modificações que alteram ou expandem recursos do sistema). O termo “jailbreak” significa “fuga da prisão”, pois libera o dispositivo das limitações impostas pela Apple.



Hoje, os **jailbreaks** mais comuns usam exploits de hardware, como o bootrom **checkm8** (usado pelo Checkra1n e Palera1n), ou **exploits** userland **baseados em software**, como o **Dopamine**, que utiliza uma cadeia de exploits de software (userland + kernel + bypasses de code-signing/PPL), como o **Dopamine**, que usa vários kernel-exploits e bypasses de proteção.

Com o aumento da complexidade do sistema e a implementação de proteções avançadas no ecossistema da Apple, **jailbreaks simples e untethered** que ofereciam **persistência** mesmo após reiniciar ou desligar o dispositivo **tornaram-se raros**. Atualmente, predominam os jailbreaks semi-untethered e semi-tethered, que exigem a reexecução do exploit a cada reinicialização para manter o acesso ao dispositivo.

Jailbreak: Métodos

Rootless Jailbreak

- Armazena dados e modificações em **/var/jb**, sem alterar o sistema raiz (/)
- Pode ter **incompatibilidade com tweaks desatualizados**, embora isso esteja se tornando menos comum

Rootful Jailbreak

- Monta e escreve diretamente em **/**, com permissões root completas
- Ideal para modificações profundas, porém mais invasivo e detectável. Isso concede acesso total de superusuário (root) ao sistema, permitindo **modificações profundas** que afetam diretamente o sistema operacional principal localizado no diretório /.

Usa ElleKit como motor de injeção de tweaks na grande maioria dos jailbreaks rootless modernos

Jailbreak: Tipos de Jailbreak - Untethered

Untethered

Esse tipo de Jailbreak precisa ser **executado apenas uma vez**. Mesmo após reiniciar ou desligar o dispositivo, **o Jailbreak permanece ativo**. No entanto, esse método só está disponível para versões mais antigas do iOS. Atualmente, não há novos Jailbreaks Untethered sendo lançados.

- **JailbreakMe** – Compatível com iOS 4.x
- **Absinthe** – Compatível com iOS 5.x
- **evasi0n** – Compatível com iOS 6.x a 7.x
- **PanGu9** – Compatível com iOS 9.2 a 9.3.3



Jailbreak: Tipos de Jailbreak - Tethered

Tethered

Após a realização desse tipo de jailbreak, o **dispositivo precisará estar conectado** a um computador **sempre que for desligado ou reiniciado**, para que possa ser inicializado novamente. Não é prático para usuários comuns e costuma ser utilizado apenas por pesquisadores de segurança iOS, pois oferece maior controle sobre o processo de inicialização e permite que falhas sejam reproduzidas de forma controlada. Exemplos:

- **Blackra1n** – Compatível com iOS 3.x até 4.x
- **Redsn0w** – Compatível com iOS 3.x, 4.x e algumas versões do 5.x



Jailbreak: Tipos de Jailbreak - Semi-Untethered

Semi-Untethered

Nesse tipo de Jailbreak, é necessário **re-executar o exploit** **sempre que o dispositivo for reiniciado ou desligado**. A vantagem é que o processo pode ser feito diretamente no aparelho, por meio de um aplicativo.

- **Odyssey** – iOS 13.0 a 13.7
- **Taurine** – iOS 14.0 a 14.8.1
- **Dopamine** – iOS 15.0 a 16.5 (todos os dispositivos), iOS 16.5.1 (A14 e anteriores e M1) e iOS 16.6 a 16.6.1 (A11 e anteriores).

Jailbreak: Tipos de Jailbreak - Semi-Tethered

Semi-Tethered

Assim como o semi-untethered, esse tipo também **requer re-execução do exploit após reiniciar ou desligar o dispositivo**. No entanto, o processo deve ser feito sempre com o **auxílio de um computador** ao invés de um aplicativo instalado no dispositivo.

- **Checkra1n** – Compatível com iOS 12.x a 14.x
- **Palera1n** – Compatível com iOS 15.x a 16.x (baseado no *exploit* checkm8)

OVER SECURITY INTELLIGENCE

Jailbreak na Prática



Jailbreak na prática

1. Desabilite as atualizações

   ➔ Acesse: Ajustes > Geral > Atualizações Automáticas

1. Identifique o modelo e iOS do dispositivo:

   ➔ Acesse: Ajustes > Geral > Sobre

- Modelo (ex: iPhones Xs)
- Versão do iOS (ex: 16.5.1)



Jailbreak na prática

Verifique compatibilidade com jailbreaks disponíveis:

https://appledb.dev/

https://ios.cfw.guide/



From	To	Jailbreak
16.6	18.5	N/A
15.0	16.5.1	Installing Dopamine
14.0	14.8.1	Installing Taurine
13.0	13.7	Installing Odyssey
12.1.3	12.4.1	Installing unc0ver
12.0	12.1.2	Installing Chimera

Jailbreak I - Dopamine



Dopamine

- iOS 15.0 - 16.6.1
- semi-untethered jailbreak

Chip	Modelos de iPhone compatíveis
A8	iPhone 6, iPhone 6 Plus
A9	iPhone 6s, 6s Plus, iPhone SE (1ª geração)
A10	iPhone 7, 7 Plus
A11	iPhone 8, 8 Plus, iPhone X
A12	iPhone XR, XS, XS Max
A13	iPhone 11, 11 Pro, 11 Pro Max
A14	iPhone 12, 12 Mini, 12 Pro, 12 Pro Max
A15	iPhone 13, 13 Mini, 13 Pro, 13 Pro Max, iPhone SE (3ª geração)
A16	iPhone 14 Pro, 14 Pro Max
M1	iPad Pro (2021), Macs (Apple Silicon)
M2	iPad Pro (2022), Macs

https://ellekit.space/dopamine/

Apresentação do @opa334dev - Dopamine deep dive

Jailbreak III - Boot USB com palen1x



palen1x

- Ótima opção para ficar diferente do Dopamine
- A8(X) a A11* no iOS 15.0 a 18.3.2
- Necessário pendrive bootável
- Rootless e Rootful

obs1: Nos A11, você deve desativar sua senha

obs2: O boot não é recomendado em PCs com CPU “*Ryzen*”

Chip	Modelos de iPhone compatíveis
A8	iPhone 6, iPhone 6 Plus
A9	iPhone 6s, 6s Plus, iPhone SE (1ª geração)
A10	iPhone 7, 7 Plus
A11	iPhone 8, 8 Plus, iPhone X

Jailbreak III - Boot USB com palen1x



palen1x

1 - Pendrive bootável

- Rufus https://rufus.ie/
- Ventoy2 https://github.com/ventoy/Ventoy/releases



Rufus

2 - Imagem ISO Palen1x:

https://github.com/palera1n/palen1x/releases



Ventoy2

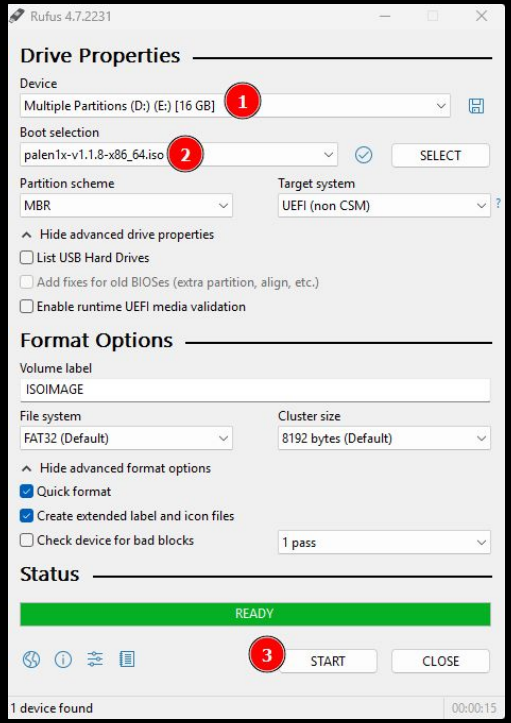
Jailbreak III - Boot USB com palen1x

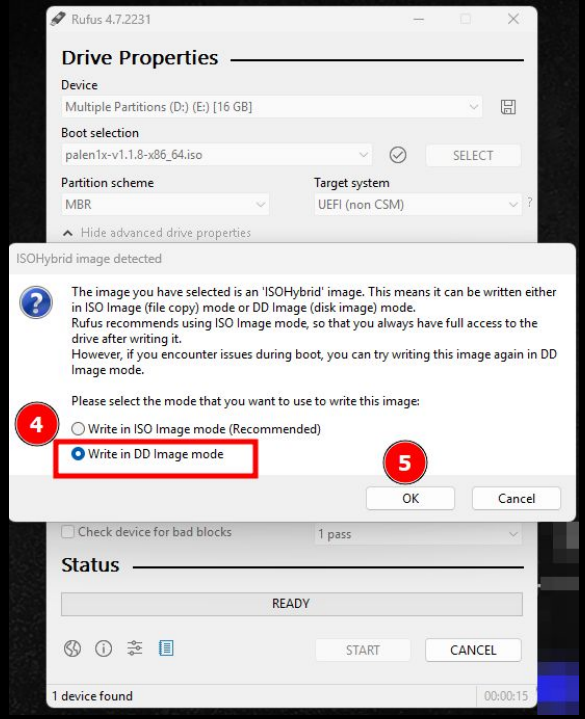


palen1x



Rufus

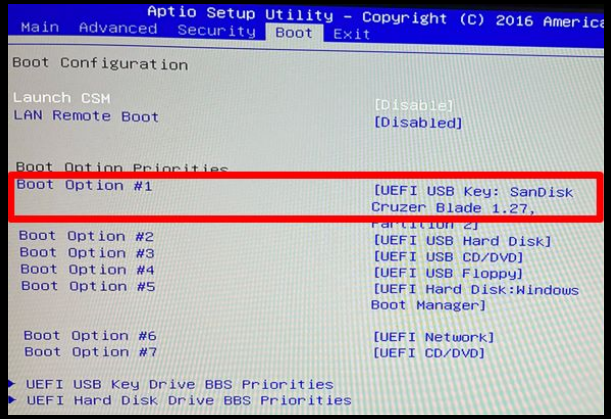




Jailbreak III - Boot USB com palen1x

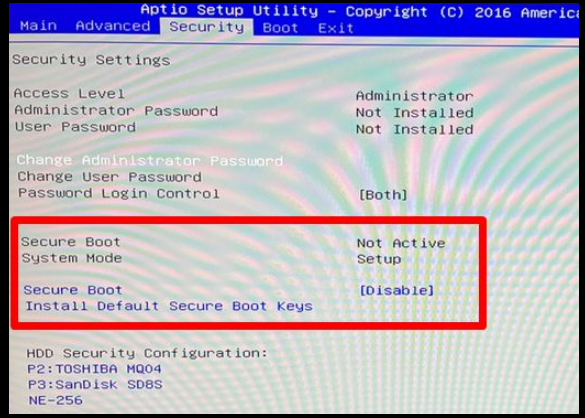


palen1x



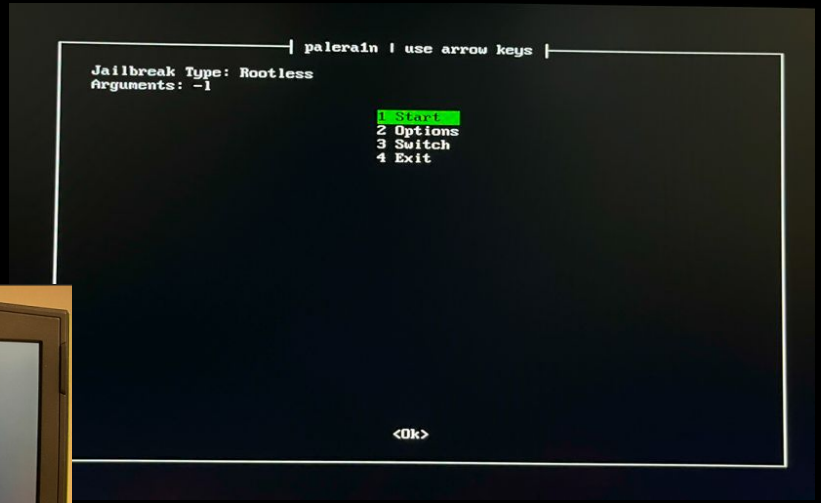
2 - Conferir ordem do boot

1 - Desabilitar Secure Boot



Jailbreak III - Boot USB com palen1x

palen1x







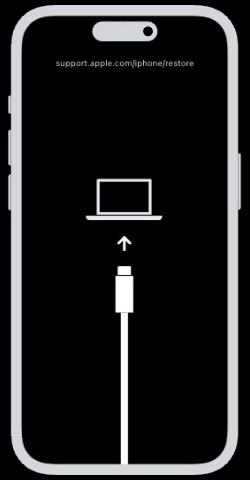
Jailbreak III - Boot USB com palen1x



palen1x

MODO DFU (Device Firmware Update)

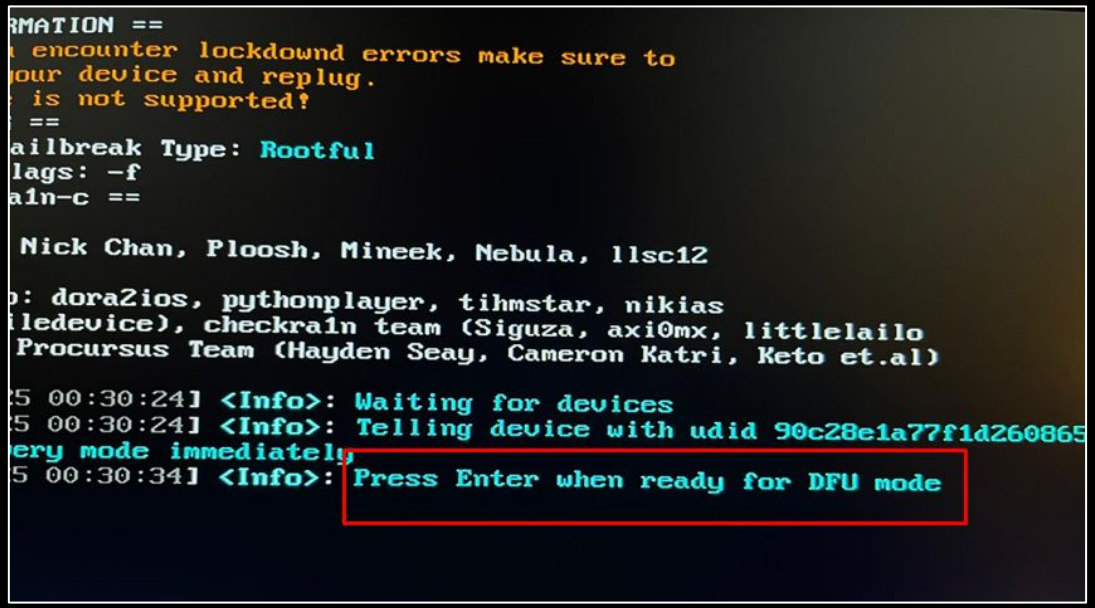
- É o método mais profundo de recuperação
- O processo depende de cada modelo
- Em todos serão com cabo plugado e com o dispositivo desligado
- O palen1x reinicia caso estiver ligado
- Modo DFU não exibe tela (não é restore)
- Se aparecer a logo da apple ou a tela *restore = fail*
- https://support.apple.com/pt-br/118106

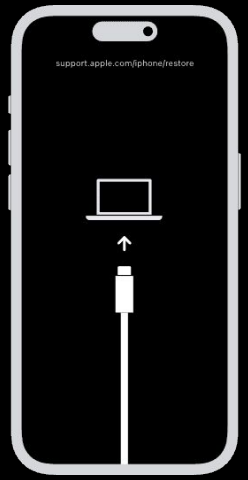


Jailbreak III - Boot USB com palen1x



palen1x



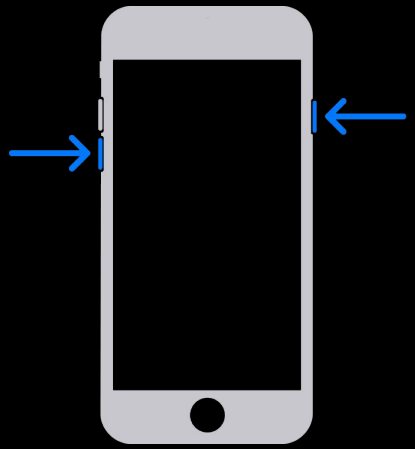


Jailbreak III - Boot USB com palen1x

palen1x

Exemplo do procedimento DFU no A10 (iPhone 7):

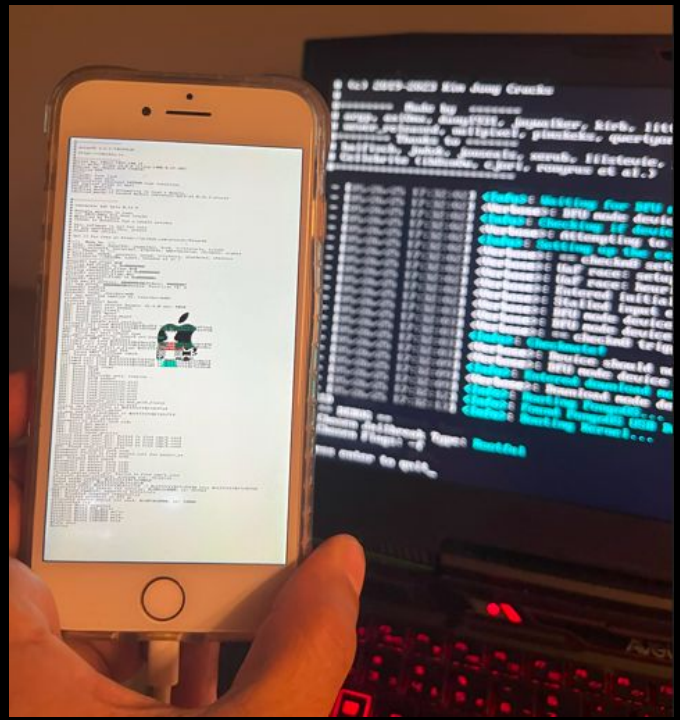
1. Botões: “ligar” + “diminuir volume”
2. Após 5s solte apenas o botão “ligar”
3. Aguardar mensagem DFU no computador (até 5s)



Jailbreak III - Boot USB com palen1x



palen1x



No exemplo:

- A10 (iPhone 7)
- iOS 15.7.5
- Palen1x
- USB bootável
- Rootful

OVER SECURITY INTELLIGENCE

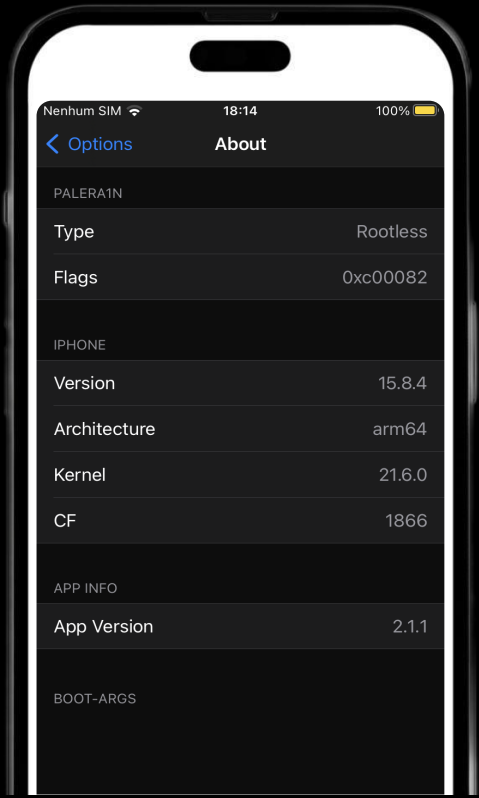
Jailbreak III - Boot USB com palen1x

palen1x

rootful



rootless



Jailbreak - tryharder

PHOENIX

kok3shi9

Checkra1n

PanGu

Cydia

LiberiOS

liberTV

unc0ver

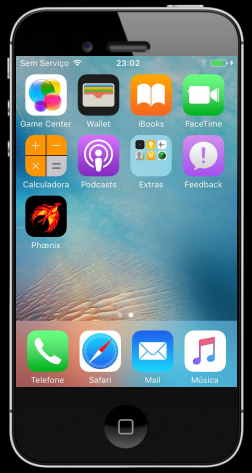
- The State of iOS Jailbreaking in 2025.pdf
- alfiecg24/Trigon: Deterministic kernel exploit based on CVE-2023-32434.
- https://alfiecg.uk/2025/03/01/Trigon.html

Root Hide





iPhone 4s



iPhone 4s

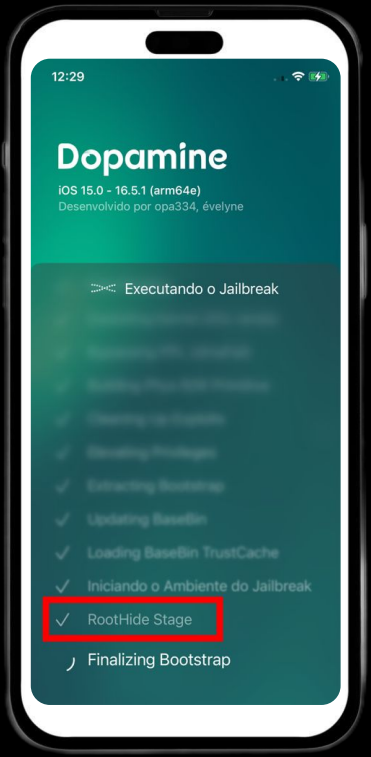


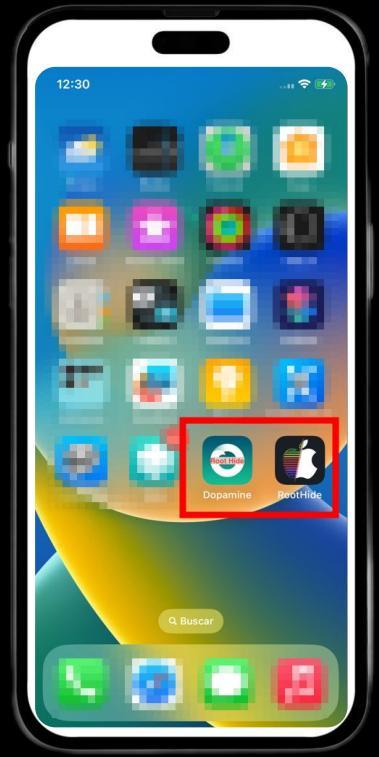
iPhone XS

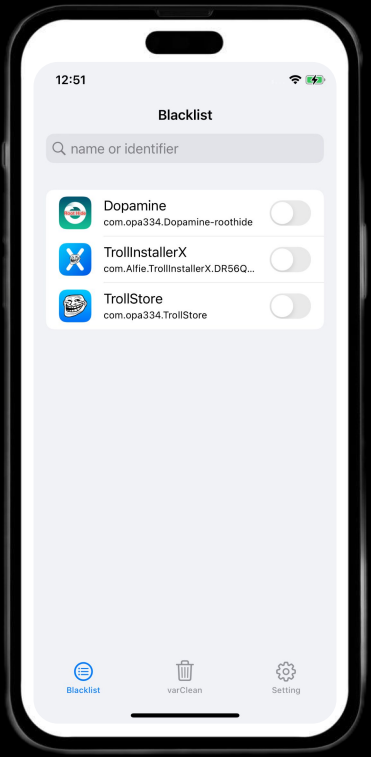
OVER SECURITY INTELLIGENCE

Diferença na prática rootless e rootful

Alternativa Dopamine: Root Hide







OVER SECURITY INTELLIGENCE

Proteções Mobile



Detecção de Jailbreak em iOS: Técnicas Comuns

A **detecção de jailbreak** é uma técnica usada para dificultar a execução do app em dispositivos modificados, bloqueando ferramentas e métodos comuns de engenharia reversa. Sozinha, essa defesa pode ser facilmente contornada, mas ao espalhar verificações por diferentes partes do código, sua eficácia aumenta e contribui para uma proteção mais robusta contra adulterações.

O tópico do MASTG de iOS Anti-Reversing Defenses, no que tange a Jailbreak, alerta sobre possíveis técnicas comuns utilizadas para detectar acesso a aplicativos vindo de dispositivos jailbroken, entre elas:

- Verificações baseadas em arquivos
- Verificações baseadas em permissões de arquivo
- Verificações baseadas em manipuladores de protocolo

Detecção de Jailbreak em iOS: Técnicas Comuns

1. Verificação de Arquivos e Diretórios

O app verifica a existência de arquivos comuns em dispositivos com jailbreak, como:

- Apps: /Applications/Cydia.app, SBSettings.app, WinterBoard.app, etc.
- Ferramentas de terminal: /bin/bash, /usr/sbin/sshd, /etc/apt
- Frida e Cycript: /usr/sbin/frida-server, /usr/bin/cycript, libcycript.dylib
  Diretórios sensíveis: /private/var/lib/cydia, /var/tmp/cydia.log

```
func checkJailbreak() {
    let paths = [
        "/Applications/Cydia.app",
        "/bin/bash",
        "/usr/sbin/frida-server",
        "/usr/bin/cycript",
        "/private/var/lib/cydia"
    ]
    if paths.contains(where:
FileManager.default.fileExists(atPath:)) {
        exit(0)
    }
}
```

2. Verificação de Permissões de Escrita Fora da Sandbox

Apps iOS legítimos são restritos pela sandbox, o que impede escrita em diretórios protegidos do sistema. Um app pode tentar escrever em um caminho restrito, como */private*, para verificar se o sistema foi desbloqueado:

```
do {
    let testPath = "/private/jailbreak_test.txt"
    try "teste".write(toFile: testPath, atomically: true, encoding: .utf8)
    try FileManager.default.removeItem(atPath: testPath)
    print("Jailbreak detectado – encerrando app.")
    exit(0)  // Encerra o app imediatamente
} catch {
    print("Sistema provavelmente íntegro")
}
```

Se a escrita for bem-sucedida, é um forte indicativo de que o app está fora das restrições normais — sinal de jailbreak.

3. Verificação de Manipuladores de URL (Custom URL Schemes)

Dispositivos com jailbreak geralmente instalam apps que **registram esquemas de URL personalizados**, como cydia://. O app pode usar canOpenURL() para checar se esse tipo de URL é reconhecido:

```
if let url = URL(string: "cydia://package/com.example.pkg"),
   UIApplication.shared.canOpenURL(url) {
    print("Jailbreak detectado via URL scheme – encerrando app.")
    exit(0)
}
```

Se o sistema conseguir abrir a URL, é provável que o app Cydia (ou equivalente) esteja instalado — outro sinal claro de jailbreak.

Anti-Debugging/Hook

Soluções de **Anti-Debugging** ou **Anti-Hook**, são mecanismos usados por apps para detectar e reagir a ferramentas de depuração ou instrumentação dinâmica, como Frida e LLDB. O objetivo é dificultar a engenharia reversa, proteger dados sensíveis e impedir modificações no aplicativo em tempo de execução.

Conforme definido no MASTG-TEST-0089: Testing Anti-Debugging Detection, para verificar se um app implementa mecanismos de anti-debugging, é possível realizar testes utilizando ferramentas de depuração ou instrumentação dinâmica e o app deve reagir de alguma forma à tentativa de depuração, como por exemplo:

- Exibir um alerta ao usuário ou encerrar a execução imediatamente;
- Apagar dados sensíveis armazenados localmente; e
- Notificar um servidor backend (ex: para fins de detecção de fraude ou uso indevido).

Essas medidas visam dificultar a **análise dinâmica e a engenharia** reversa, elevando a complexidade para atacantes que tentam manipular ou inspecionar o comportamento do aplicativo.

Anti-Frida

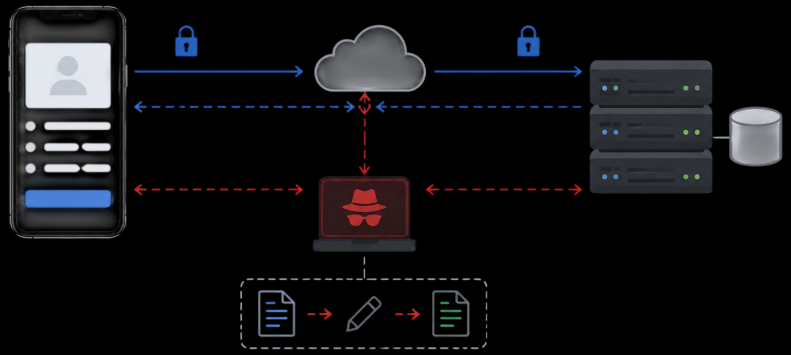
Dentre os métodos mais comuns utilizados por soluções de Anti-Frida, especificamente no contexto de detecção de Frida, a OWASP destaca as seguintes abordagens utilizadas:

- **Detecção de Artefatos no Ambiente**: busca por processos, daemons (ex: frida-server) e bibliotecas suspeitas (ex: frida-agent). Pode ser contornado com renomeação ou ocultação.
- **Monitoramento de Portas TCP**: verifica portas usadas por ferramentas de debug remoto, como a porta padrão 27042 do frida-server. Fácil de contornar, bastando apenas alterar a porta do frida-server.
- **Verificação de Protocolos de Autenticação**: testa respostas em protocolos de debug (ex: D-Bus no Android). Método eficaz, porém limitado a configurações padrão.
- **Escaneamento da Memória**: procura por strings, símbolos e bibliotecas típicas carregadas na memória do app. Mais difícil de burlar, mas não infalível.

Alguns desses métodos, como inspeção de processos ou memória, são difíceis ou impossíveis de implementar diretamente em linguagens como Swift devido a restrições do iOS, sandboxing e limitações nas APIs de baixo nível.

MitM via Proxy

Ataque **Man-in-the-Middle** (MiTM) via **Proxy** ocorre quando um atacante intercepta a comunicação entre o aplicativo mobile e a API. Utilizando um proxy de interceptação (exemplo: Burp Suite), é possível visualizar, modificar e re-enviar as requisições e respostas trocadas com a API e outros microsserviços, permitindo analisar o funcionamento da aplicação e identificar possíveis vulnerabilidades na comunicação.



TLS Pinning

TLS Pinning é uma **técnica de segurança** usada em aplicativos para garantir que, durante uma conexão TLS, o cliente aceite apenas um certificado específico ou uma chave pública previamente confiável do servidor. Isso evita que atacantes realizem ataques *man-in-the-middle* via *proxy* e interajam diretamente com a API usando certificados falsificados ou emitidos por autoridades não confiáveis, reforçando a proteção da comunicação entre cliente e servidor.



TLS Pinning

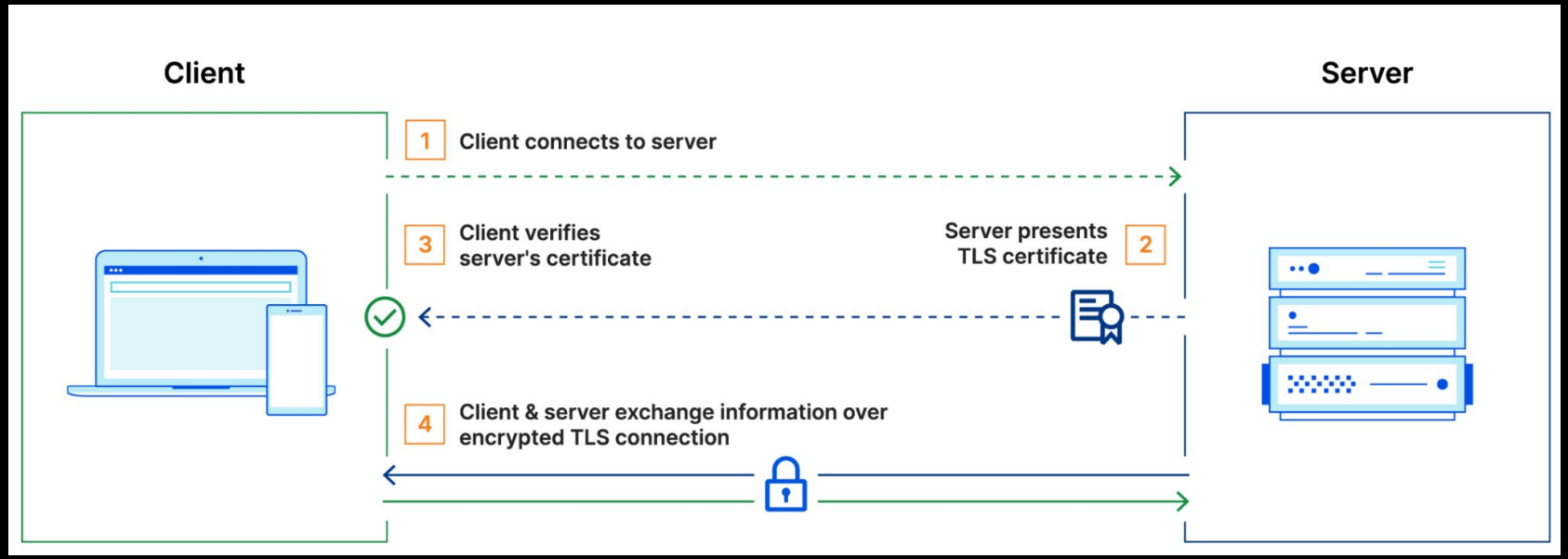
Esses conceitos de pinning e hash reforçam a segurança das conexões TLS, garantindo a autenticidade e integridade dos certificados e chaves usadas na comunicação.

Certificate Pinning: Na fixação de certificado, o aplicativo cliente é configurado para validar especificamente o certificado do servidor durante a conexão TLS. Isso garante que a comunicação seja feita somente com o servidor legítimo, aumentando a segurança contra ataques man-in-the-middle.

Public Key Pinning: Na fixação de chave pública, quando um cliente acessa um site, o servidor “fixa” sua chave pública no navegador do cliente. Em visitas futuras, o cliente verifica se a chave pública recebida corresponde à previamente fixada, assegurando a integridade e autenticidade da conexão.

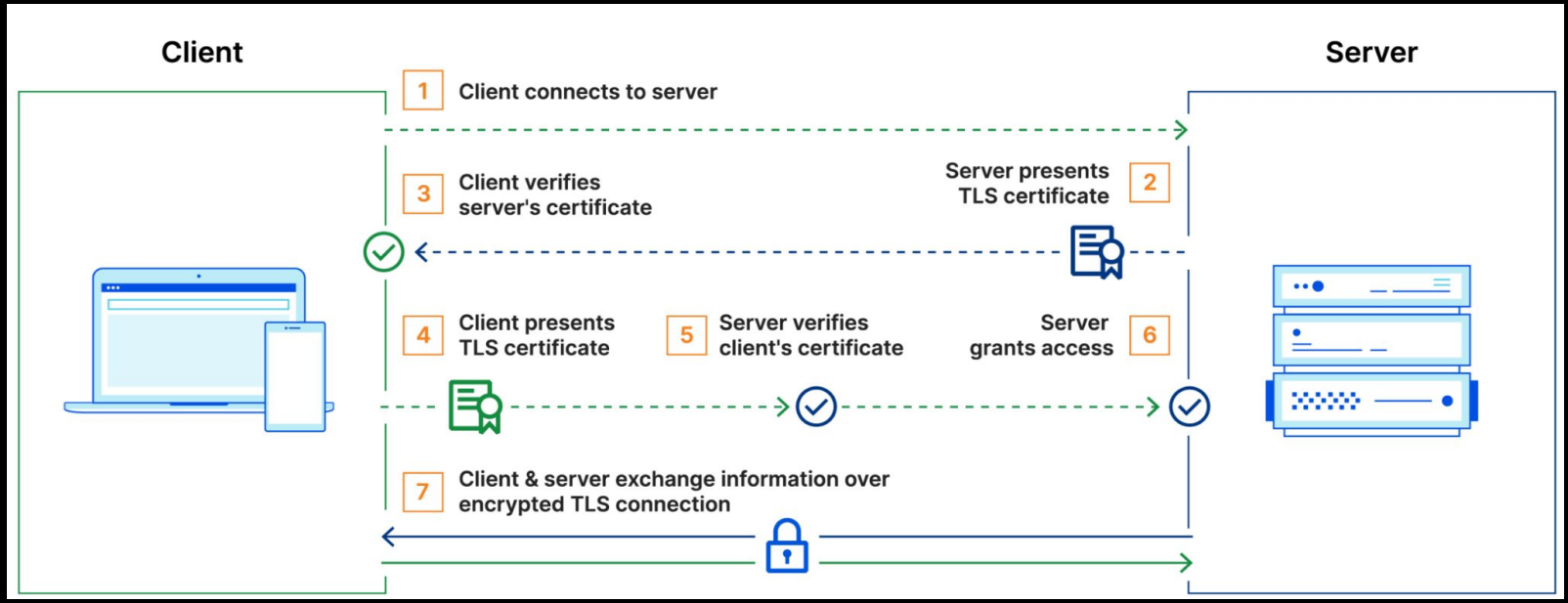
Hash: Hashes podem ser usados para representar de forma anônima certificados ou chaves públicas, protegendo informações sensíveis e dificultando o vazamento desses dados durante processos de pinning.

TLS (Transport Layer Security)



[O que é mTLS - CloudFlare]

mTLS (mutual Transport Layer Security)



[O que é mTLS - CloudFlare]

mTLS - Certificados

Certificado único para todos os clientes

- Fácil de gerenciar, ideal para ambientes internos e controlados;
- Baixa visibilidade individual e difícil revogação seletiva;
- Alto risco em caso de vazamento, pois compromete todos os clientes.

Certificado individual por cliente/dispositivo

- Maior segurança e rastreabilidade por cliente;
- Comprometimento isolado e revogação seletiva;
- Requer infraestrutura de PKI mais robusta e maior complexidade de gestão.

mTLS na visão do BACEN

O Manual de Segurança do Pix v3.7, elaborado pelo Banco Central, estabelece que todas as APIs utilizadas pelos participantes ao longo da cadeia de funcionalidades do Pix devem assegurar que apenas os aplicativos oficiais das instituições acessem seus sistemas, bloqueando robôs e scripts automatizados, além de prevenir ataques do tipo **man-in-the-middle** por meio de técnicas como **mTLS**, a fim de que toda a comunicação seja cifrada e ocorra apenas com as APIs e sistemas desejados, evitando qualquer interceptação ou manipulação.

É exigido que esses aplicativos incorporem **mecanismos robustos contra engenharia reversa**, descompilação, adulteração de código e modificação de parâmetros de segurança. Por fim, ele também reforça que a proteção das APIs e sistemas relacionados ao Pix deve estar concentrada majoritariamente no lado servidor, sem depender exclusivamente das defesas implementadas nos dispositivos dos usuários.

Também é necessário adotar mecanismos como *Token Bucket* para prevenir ataques de leitura excessiva de chaves Pix e limitar o número de requisições por software cliente, conforme os parâmetros do Manual Operacional do DICT. Além disso, recomenda-se o uso de medidas adicionais de segurança, como **autenticação em dois fatores, CAPTCHA** e tokens vinculados a dispositivos previamente cadastrados.

OVER SECURITY INTELLIGENCE Técnicas de Proteção em apps iOS: Ofuscação e Criptografia

Apesar de o Swift aplicar certa **ofuscação natural** através de símbolos mangled e um runtime mais fechado, e do Objective-C oferecer menos exposição estática, **essas proteções embutidas não são suficientes contra engenharia reversa avançada. Ferramentas como Frida, Ghidra, Hopper, IDA Pro e Radare2** ainda conseguem mapear, interpretar e alterar o comportamento de apps com relativa facilidade quando não há medidas adicionais.

- **Ofuscação de Nomes de Métodos**
  Métodos (selectors) ficam acessíveis em tempo de execução pela runtime Objective-C, usada também por apps Swift para interoperabilidade. Ferramentas estáticas (Hopper) e dinâmicas (Frida) exploram isso para identificar funções críticas. Renomear métodos sensíveis dificulta essa análise.

- **Criptografia de Strings**
  Dados sensíveis embutidos no código, como chaves, certificados ou tokens, devem ser criptografados para dificultar sua extração e modificação por invasores. Chaves e processos de criptografia podem ser ofuscados e realizados em runtime para maior segurança.

- **Ofuscação do Fluxo de Controle**
  Dividir e embaralhar a lógica da verificação dificulta o entendimento do fluxo e a localização dos pontos críticos por análises estáticas e dinâmicas.

Essas técnicas, combinadas com práticas como anti-debugging, integridade do binário e modularização, elevam significativamente a complexidade de bypasses baseados em engenharia reversa estática.

OVER SECURITY INTELLIGENCE

Engenharia Reversa



OVER SECURITY INTELLIGENCE

Engenharia Reversa: Estática e Dinâmica



Engenharia Reversa: Introdução

Engenharia reversa é o processo de analisar um sistema para entender seu funcionamento interno e tentar subvertê-lo. Em iOS, essa prática é fundamental para avaliar segurança, identificar vulnerabilidades ou adaptar funcionalidades.

Existem duas abordagens principais:

- **Estática**: consiste em examinar o código compilado ou os binários sem executar o aplicativo, usando ferramentas como disassemblers para entender a estrutura e lógica, utilizando ferramentas como ghidra, radare2 e hopper.
- **Dinâmica**: envolve executar o app em tempo real, monitorando seu comportamento, chamadas de função e manipulação de memória, com o auxílio de debuggers ou frameworks como *Frida*, *Objection*, LLDB, entre outros.

Ambas se complementam para permitir uma análise completa do software.

Engenharia Reversa Dinâmica



O **Frida** é uma ferramenta de instrumentação dinâmica utilizada para analisar e modificar o comportamento de aplicações em tempo de execução. Em apps mobile Android e iOS, ele permite interceptar chamadas de funções, alterar valores de variáveis, monitorar métodos internos e contornar mecanismos de segurança diretamente no runtime do aplicativo.



O **Objection** é uma ferramenta construída sobre o Frida que facilita a análise dinâmica de aplicações mobile. Ele fornece uma interface com comandos prontos para executar tarefas comuns de pentest, como explorar o armazenamento do aplicativo, ignorar mecanismos de proteção, inspecionar classes e métodos e interagir com o aplicativo em execução sem necessidade de escrever scripts complexos.

OVER SECURITY INTELLIGENCE

Frida Script deu ruim, e agora ?



Pensando como um atacante

```
[xf@Mac ~ % objection -g com.oversecurity.notes.A69W5SA6CA explore -s "ios jailbreak disable"
/Users/xf/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(


A newer version of objection is available!
You have v1.11.0 and v1.12.4 is ready for download.

Upgrade with: pip3 install objection --upgrade
For more information, please see: https://github.com/sensepost/objection/wiki/Updating

Using USB device `iPhone`
Agent injected and responds ok!
Running a startup command... ios jailbreak disable
(agent) Registering job 345889. Type: ios-jailbreak-disable


     _   _         _   _
 ___| |_|_|___ ___| |_|_|___ ___
| . | . | | -_|  _|  _| | . |   |
|___|___| |___|___|_| |_|___|_|_|
      |___|(object)inject(ion) v11.0

     Runtime Mobile Exploration
        by: @leonjza from @sensepost

[tab] for command suggestions
....oversecurity.notes.A69W5SA6CA on (iPhone: 15.8.6) [usb] #
```



Pensando como um atacante

```
[xf@Mac ~ % frida -U --codeshare liangxiaoyi1024/ios-jailbreak-detection-bypass -f com.oversecurity.notes.A69W5SA6CA
     ____
    / _  |   Frida 16.2.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to iPhone (id=0b6028bcd73834009988e8ca38aeb4326e78c4ba)
Spawned `com.oversecurity.notes.A69W5SA6CA`. Resuming main thread!
[iPhone::com.oversecurity.notes.A69W5SA6CA ]-> Process terminated
[iPhone::com.oversecurity.notes.A69W5SA6CA ]->

Thank you for using Frida!
```



Pensando como um atacante

```
xf@Mac ~ % frida -U --codeshare patali09/ios-jailbreak-detection-bypass -f com.oversecurity.notes.A69W5SA6CA
     ____
    / _  |   Frida 16.2.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to iPhone (id=0b6028bcd73834009988e8ca38aeb4326e78c4ba)
Spawned `com.oversecurity.notes.A69W5SA6CA`. Resuming main thread!
[iPhone::com.oversecurity.notes.A69W5SA6CA ]-> fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
Process terminated
[iPhone::com.oversecurity.notes.A69W5SA6CA ]->

Thank you for using Frida!
```



Pensando como um atacante

```
xf@Mac ~ % frida -U --codeshare sdcampbell/ios-jailbreak-bypass -f com.oversecurity.notes.A69W5SA6CA
     ____
    / _  |   Frida 16.2.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to iPhone (id=0b6028bcd73834009988e8ca38aeb4326e78c4ba)
Spawning `com.oversecurity.notes.A69W5SA6CA`...
Bypassing Jailbreak detection checks
[Jailbreak Detection Bypass] success
Spawned `com.oversecurity.notes.A69W5SA6CA`. Resuming main thread!
[iPhone::com.oversecurity.notes.A69W5SA6CA ]-> [Jailbreak Detection Bypass] success
Process terminated
[iPhone::com.oversecurity.notes.A69W5SA6CA ]->

Thank you for using Frida!
```



Pensando como um atacante

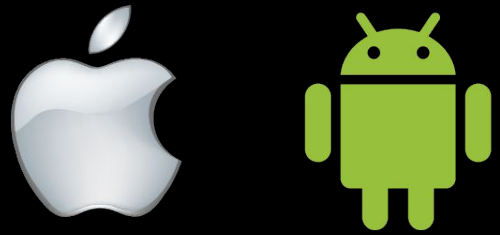
```
[xf@Mac ~ % frida -U -l liangxiaoyi1024-ios-jailbreak-detection-bypass.js -l patali09-ios-jailbreak-detection-bypass.js -l ios-jailbreak-bypass.js -f com.oversecurity.notes.A69W5SA6CA

     ____
    / _  |   Frida 16.2.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to iPhone (id=0b6028bcd73834009988e8ca38aeb4326e78c4ba)
Spawning `com.oversecurity.notes.A69W5SA6CA`...
Bypassing Jailbreak detection checks
[Jailbreak Detection Bypass] success
Spawned `com.oversecurity.notes.A69W5SA6CA`. Resuming main thread!
[iPhone::com.oversecurity.notes.A69W5SA6CA ]-> fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
fopen: bypassing/etc/passwd
[Jailbreak Detection Bypass] success
Process terminated
[iPhone::com.oversecurity.notes.A69W5SA6CA ]->

Thank you for using Frida!
```



Engenharia Reversa: Android vs iOS



A engenharia reversa em **Android** é **mais acessível** e de alto nível, pois os apps são distribuídos em bytecode (DEX) que pode ser facilmente convertido em código Java/Kotlin legível usando ferramentas como jadx, permitindo análise direta da lógica, o que é muito mais intuitivo do que trabalhar diretamente com o código smali.

Já no **iOS**, o processo é mais complexo e de **baixo nível**, pois os aplicativos são distribuídos como binários Mach-O compilados para a arquitetura ARM64, sem código-fonte facilmente recuperável. Isso exige análise detalhada de assembly com ferramentas como Ghidra, Radare2, IDA Pro ou Hopper.

Engenharia Reversa em app Objective-C e Swift



Apps iOS em Objective-C mantêm muitos símbolos legíveis no binário devido ao runtime dinâmico, facilitando a engenharia reversa mesmo sem ofuscação pesada.



Já apps em Swift usam símbolos mangled e um runtime mais fechado, aplicando uma ofuscação natural que dificulta identificar nomes e funções sem ferramentas específicas.

Assim, a engenharia reversa em Swift é geralmente mais complexa e requer mais esforço para recuperar a lógica do app, enquanto em Objective-C a estrutura do código é mais acessível para análise.

Swift Symbols

Symbols (símbolos) são nomes de funções, métodos, classes, variáveis, etc., usados durante o desenvolvimento.

Swift symbols são os símbolos gerados pelo compilador Swift. Eles geralmente aparecem "mangled", ou seja, codificados em strings complexas como:

Mangled:

```
_$s9notesover13AntiJailbreakC11isJailBrokenSbZ
```

Demangled:

```
notesover.AntiJailbreak.isJailBroken() -> Bool
```

Você pode "demanglar" esses símbolos usando a ferramenta SwiftDemangle.

Em engenharia reversa de apps iOS, esses símbolos ajudam a identificar funções, classes e lógica da aplicação, mesmo sem ter acesso ao projeto original em Xcode.

Arquitetura ARM64

A arquitetura ARM64 (também chamada AArch64) é usada na maioria dos dispositivos móveis modernos, incluindo iPhones.

Essa arquitetura define o **conjunto de instruções** e o **funcionamento interno** do processador, determinando como ele executa operações fundamentais como cálculos matemáticos, gerenciamento de fluxo de execução, chamadas de funções e acesso eficiente à memória.



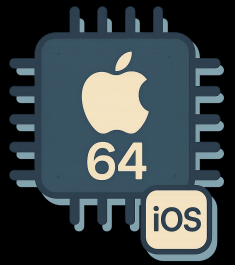
Mais informações: https://developer.arm.com/

Arquitetura ARM64 - Registradores

A arquitetura ARM64 possui **31 registradores genéricos** de 64 bits, chamados x0 a x30, usados para **armazenar valores temporários** durante a execução dos programas.

Para **operações de 32 bits**, utilizamos a versão menor desses registradores, chamados w0 a w30, que correspondem à metade inferior dos registradores xN.

Apps iOS (IPA) de arquitetura ARM64 suportam operações tanto de 32 quanto de 64 bits. Funções que retornam valores pequenos, como booleanos, podem ser manipuladas usando apenas os 32 bits inferiores, operar sobre os registradores wN é suficiente para garantir que a modificação funcione corretamente.



Exemplos de instruções importantes

A seguir as principais instruções, que você deve prestar atenção durante uma análise reversa estática em assembly, a fim de contornar possíveis proteções client-side.

1. bl — Branch with Link (chamada de função)

- **Instrução original:** chama a função isJailbroken que verifica jailbreak, o resultado da função é salvo em **w0**, retornando 1 se jailbreak for detectado e 0 caso contrário.

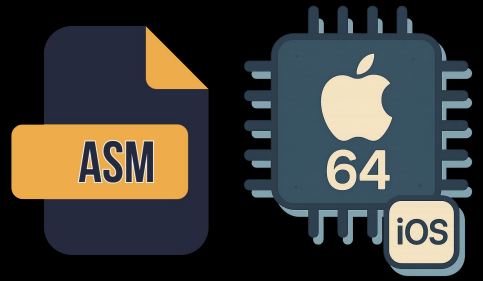
  bl isJailbroken

- **Instrução modificada para bypass:**

  mov w0, #0 ; sobrescreve a chamada e já retorna falso direto (0) no registrador w0

Mesmo sem chamar isJailbroken(), o app ainda espera um valor em w0. Ao usar mov w0, #0, simulamos que a função foi chamada e retornou false, mantendo o fluxo normal da execução.

Poderíamos também modificar para nop (instrução de "não operação"), mas, embora funcione em alguns casos, o valor em w0 fica indefinido, podendo causar comportamento inesperado no app.



Exemplos de instruções importantes para Bypass

2. mov — Move valor para um registrador

- **Instrução original:** pode ser usado para definir um valor antes de comparação, por exemplo neste caso após encontrar no dispositivo a presença de um determinado caminho que indique jailbreak, a solução atribui o valor 1 (true) em w0:

  mov w0, #1 ; exemplo: retorna verdadeiro (jailbreak detectado)

- **Instrução modificada para bypass:**

  mov w0, #0 ; força retorno falso 0 (não jailbroken)





Exemplos de instruções importantes para Bypass

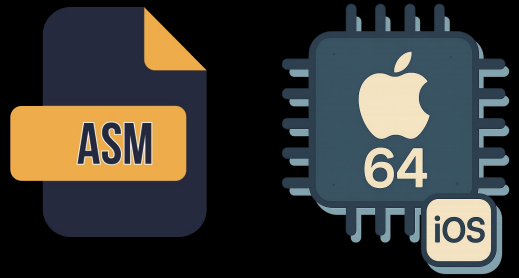
3. cmp — Compara valores

- **Original:** compara o valor retornado em w0 com 0. Se for diferente, considera que o jailbreak foi detectado e pula para uma função de encerramento ou bloqueio do app.

  cmp w0, #0
  b.ne exitApp ; pula para sair do app se w0 != 0 (jailbreak detectado)

- **Instrução modificada para bypass:**

  cmp w0, #0
  b.eq exitApp ; agora só encerra o aplicativo se w0 == 0 (sem jailbreak)



Exemplos de instruções importantes para Bypass

4. tbz — Verificação típica

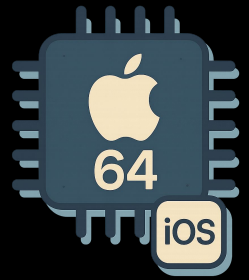
- **Original:** compara o valor retornado em w0 com 0. Se for diferente, considera que o jailbreak foi detectado e pula para uma função de encerramento ou bloqueio do app.

```
bl    _isJailbroken          ; chama função que retorna 0 (sem jailbreak) ou 1 (jailbreak)
tbz   w0, #0, continue_app   ; se bit 0 de w0 for 0 (sem jailbreak), co[illegible]a o app
b     exitApp                ; senão, pula para sair do app (jailbreak detectad
```



- **Instrução modificada para bypass:**

```
bl   _isJailbroken
tbz  w0, #1, continue_app   ; agora testa o bit 1 ao invés do bit 0
b    exitApp                ; só sai do app se bit 1 for zero (lógica invertida)
```



Engenharia Reversa Estática: Ghidra



Ghidra Software Reverse Engineering Framework

https://github.com/NationalSecurityAgency/ghidra

Ghidra é uma ferramenta gratuita e open source de engenharia reversa, principalmente estática, desenvolvida pela NSA. Oferece desassemblador, decompilador e suporte a várias arquiteturas, com interface gráfica e recursos colaborativos. Funciona em Windows, Linux e MacOS.

Você vai precisar instalar o JDK 21 64-bit para usar o GHIDRA

Obs: No MacOS compile os componentes nativos:

```
cd support/gradle

./gradlew buildNatives
```

Install

To install an official pre-built multi-platform Ghidra release:

- Install JDK 21 64-bit
- Download a Ghidra release file
  - **NOTE:** The official multi-platform release file is named `ghidra_<version>_<release>_<date>.zip` which can be found under the "Assets" drop-down. Downloading either of the files named "Source Code" is not correct for this step.
- Extract the Ghidra release file
- Launch Ghidra: `./ghidraRun` (`ghidraRun.bat` for Windows)
  - or launch PyGhidra: `./support/pyGhidraRun` (`support\pyGhidraRun.bat` for Windows)

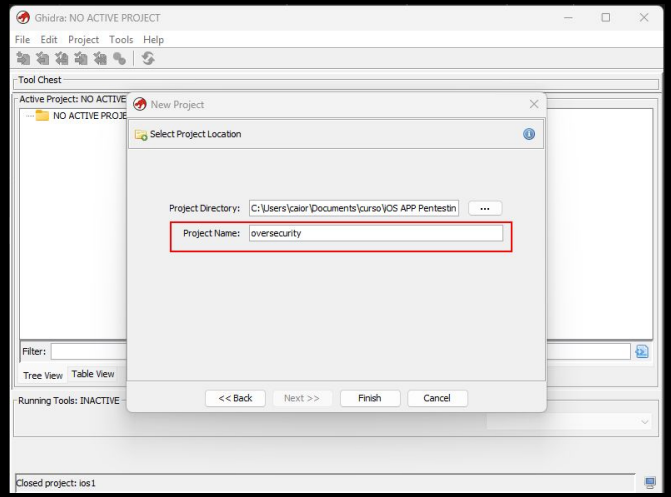
For additional information and troubleshooting tips about installing and running a Ghidra release, please refer to the Getting Started document which can be found at the root of a Ghidra installation directory.

https://adoptium.net/temurin/releases

Engenharia Reversa Estática: Ghidra

Após abrir o Ghidra executando o script (ghidraRun para linux ou ghidraRun.bat para windows10), crie um projeto seguindo os passos:

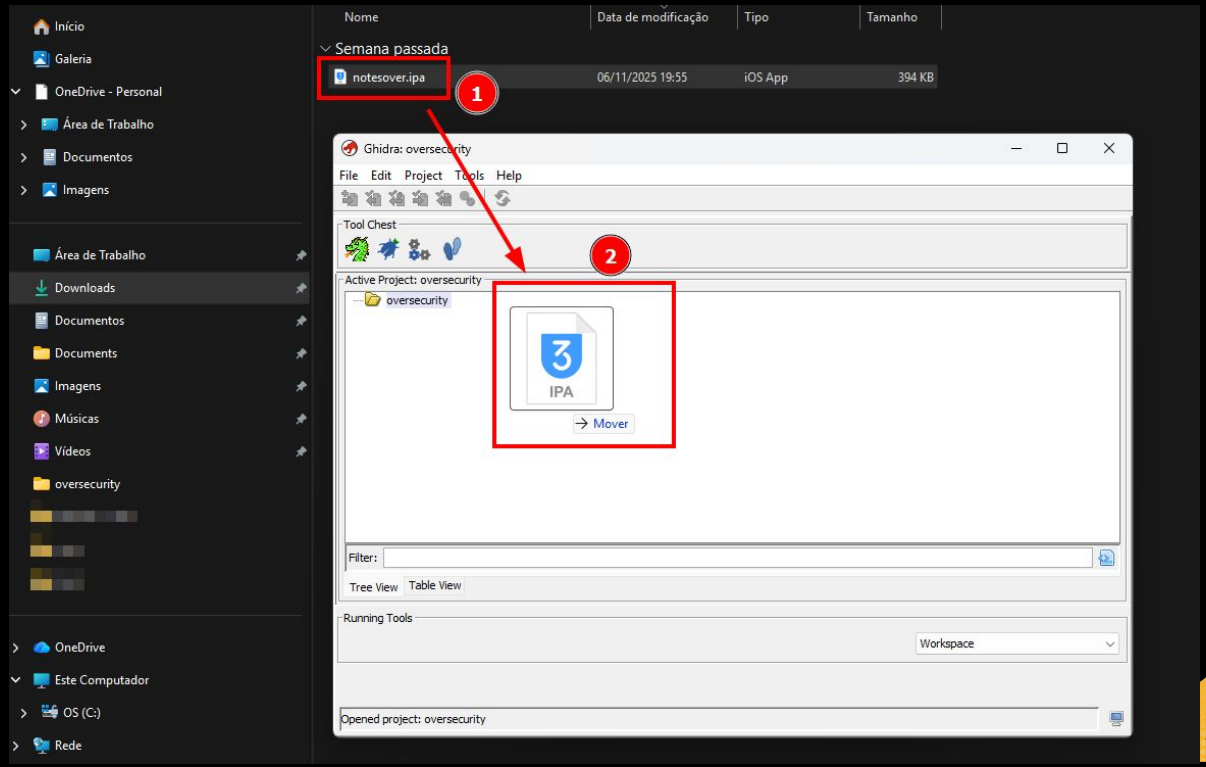
1 - Crie um novo projeto (file>new project)
2 - Selecione “Non-shared Project”
3 - Coloque o nome do projeto “oversecurity”



GHIDRA

Engenharia Reversa Estática: Ghidra

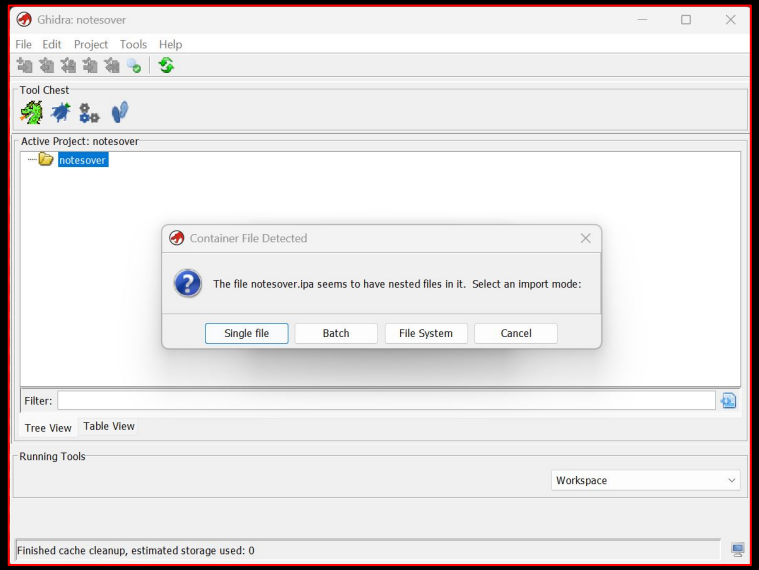
4 - Arraste o arquivo .IPA (1) para dentro do novo projeto do Ghidra(2)



GHIDRA

Engenharia Reversa Estática: Ghidra

Resumo das opções para importar .ipa no Ghidra:



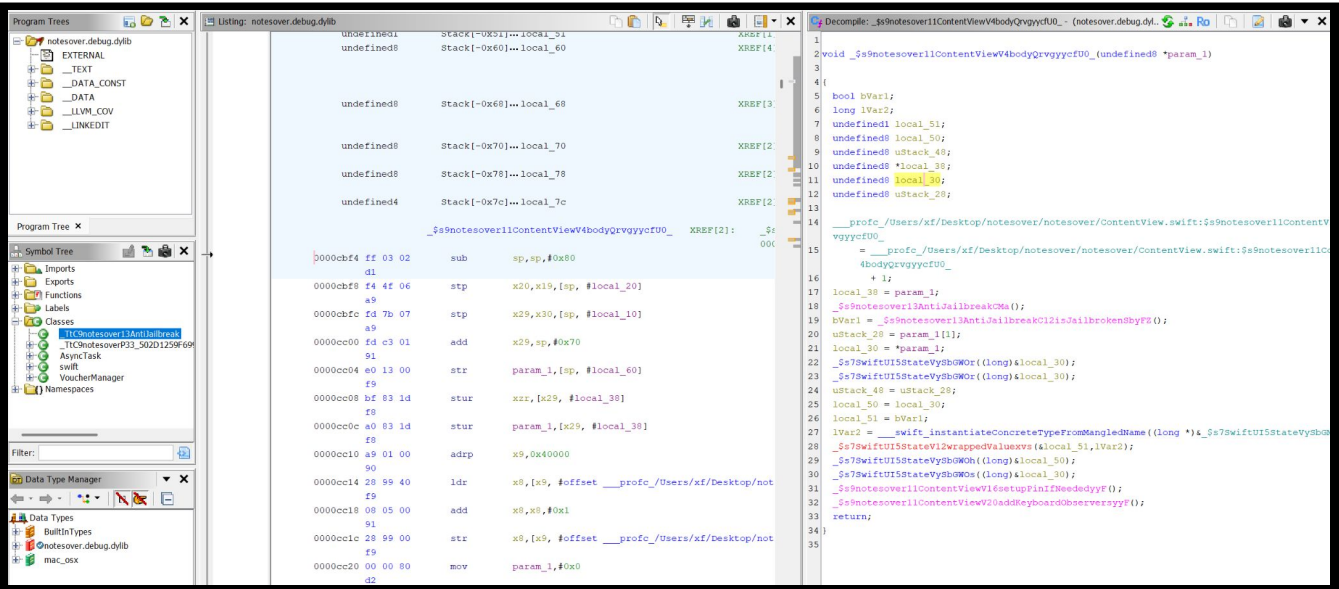
- **Single File:** importa o .ipa como um arquivo único bruto, sem extrair nada. Geralmente não é útil porque o código está dentro dos arquivos internos, não no pacote inteiro, porém pode valer a pena para buscar padrões gerais ou cabeçalhos.

- **Batch:** importa automaticamente todos os arquivos dentro do .ipa, criando várias entradas para analisar múltiplos binários e bibliotecas de uma vez. É prático para importar rapidamente tudo sem escolher individualmente.

- **File System:** monta o .ipa como um sistema de arquivos para você navegar e escolher os arquivos específicos para importar. É uma boa opção para analisar o binário principal do app e bibliotecas relevantes, focando no que interessa.

GHIDRA

Engenharia Reversa Estática: Ghidra



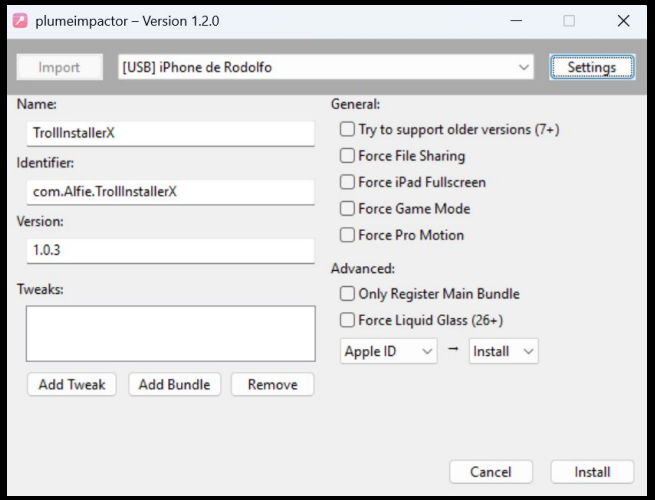
- No painel esquerdo, é exibida uma árvore de símbolos que lista todos os símbolos detectados no binário.
- No painel central, é exibido o disassembly do binário, mostrando as instruções em linguagem de máquina (assembly).
- No painel direito, é exibida a descompilação do binário para uma linguagem semelhante a C, que auxilia na compreensão da lógica e do fluxo do programa. No entanto, essa representação não corresponde ao código-fonte original em Swift.
- As modificações no binário devem ser realizadas diretamente no código assembly exibido no painel central.

Jailbreak I - Dopamine (PlumeImpactor) - NEW

Use o PlumeImpactor para instalar o aplicativo notesover.ipa



PlumeImpactor



PlumeImpactor: https://github.com/khcrysalis/PlumeImpactor

Pré requisitos:



iTunes (logar) https://support.apple.com/en-us/106372

Challenge 01 - notesover - parte 1



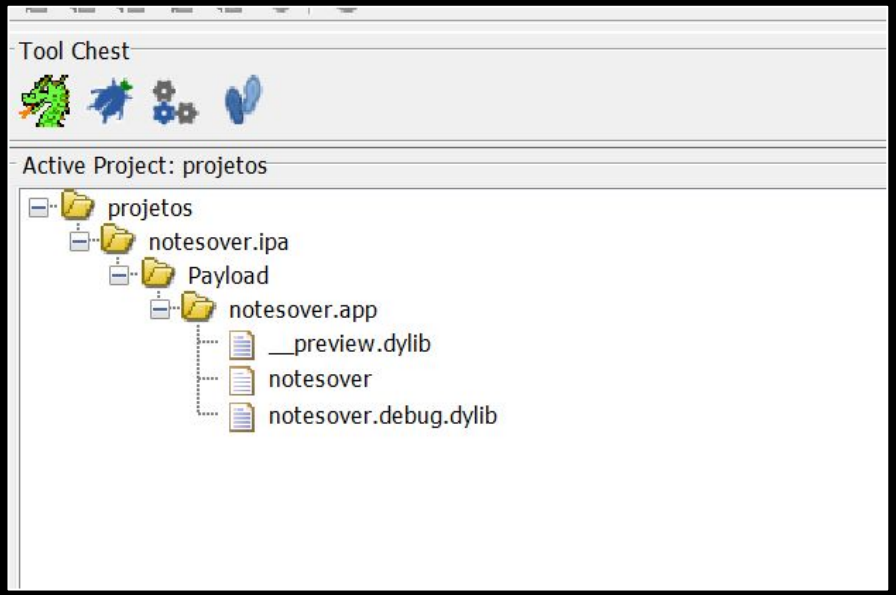
Agora vamos para o desafio!

Você possui uma versão do aplicativo de anotações que funciona offline. Ao abri-lo, o app detecta que seu dispositivo está com Jailbreak e, por isso, encerra em instantes.

Seu desafio é encontrar uma forma de burlar essa proteção através de engenharia reversa estática com Ghidra.

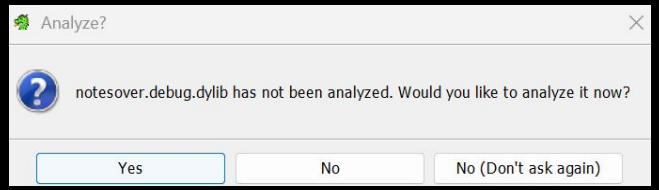
Challenge 01 - notesover - Ghidra

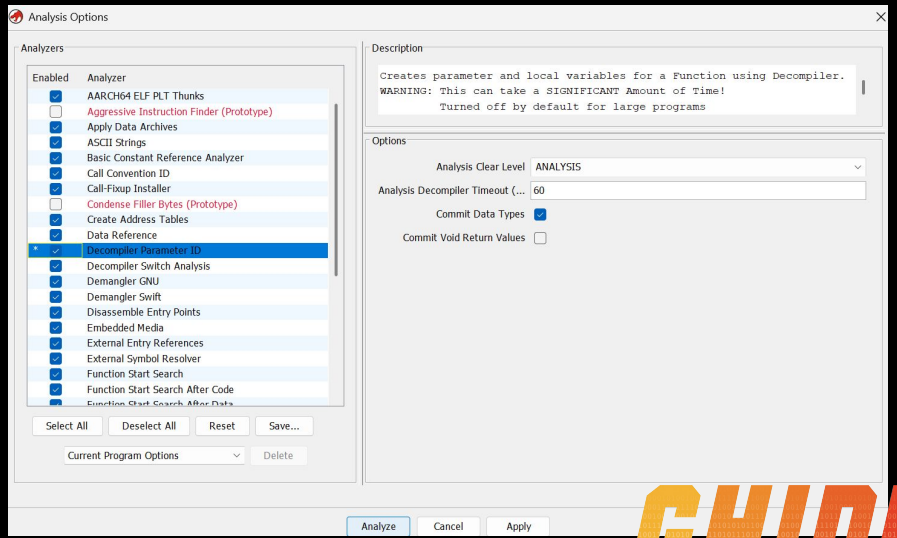
Ao importar o arquivo .ipa para o Ghidra em **modo Batch,** foram identificados três binários dentro do diretório Payload/notesover.app: o executável principal da aplicação, denominado **notesover,** e duas bibliotecas dinâmicas, sendo elas a **__preview.dylib**, relacionada a funcionalidades de visualização de conteúdo, como imagens ou vídeos, e a **notesover.debug.dylib.**



Challenge 01 - notesover - Ghidra

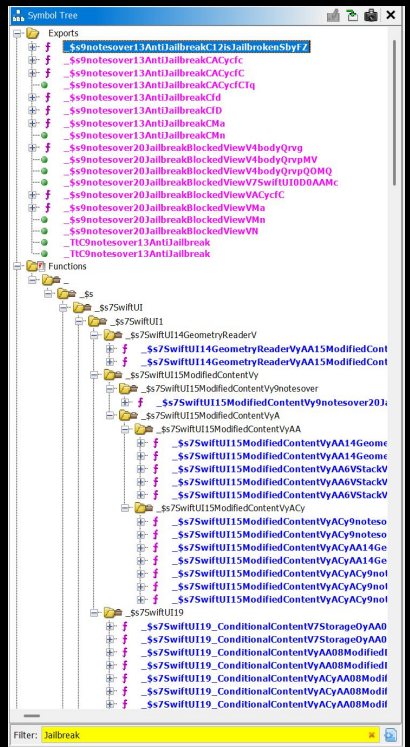
Ao clicar em qualquer um dos três binários para iniciar a análise, será exibido um aviso sobre a análise automática no Ghidra, no qual é necessário clicar em "Yes". Você verá que o Ghidra já selecionou automaticamente vários analisadores importantes para melhorar o disassembly e a desmontagem do binário. No entanto, é essencial também selecionar o analisador "Decompiler Parameter ID", pois ele contribui significativamente para uma análise mais precisa dos parâmetros utilizados nas funções durante a decompilação.





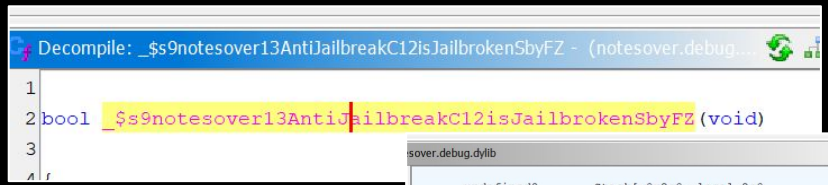
Challenge 01 - notesover - Ghidra

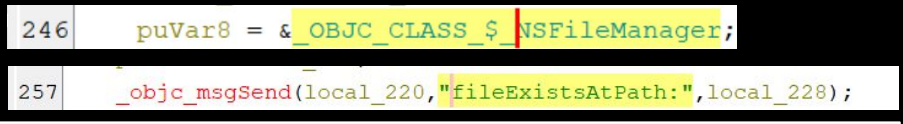
Ao analisar os binários e aplicar filtros na árvore de símbolos do painel esquerdo, em busca de classes e métodos relativos a "Jailbreak", descobrimos que a lógica da solução de Anti-Jailbreak está no binário notesover.debug.dylib. E também podemos ver um método isJailbroken da classe Anti-Jailbreak sendo exportado, em formato mangled, com o nome "_$s9notesover13AntiJailbreakC12isJailbrokenSbyFZ":

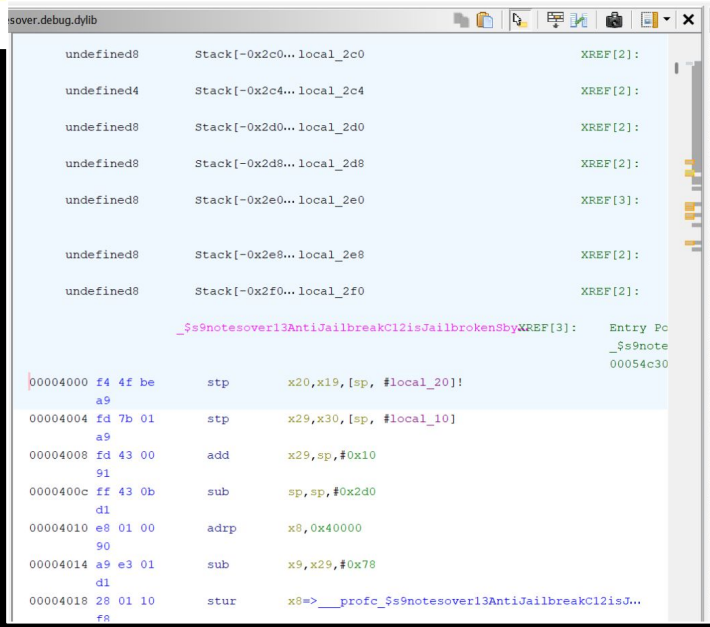


Challenge 01 - notesover - Ghidra

Clicando nele, podemos analisar o método *isJailbroken*() e verificar que está sendo utilizado o método *fileExists(atPath:)* da classe *NSFileManager* para detectar a existência de caminhos de arquivos e diretórios suspeitos que indicam *jailbreak* no dispositivo. Também é possível confirmar que o retorno esperado para o método é *boolean*.





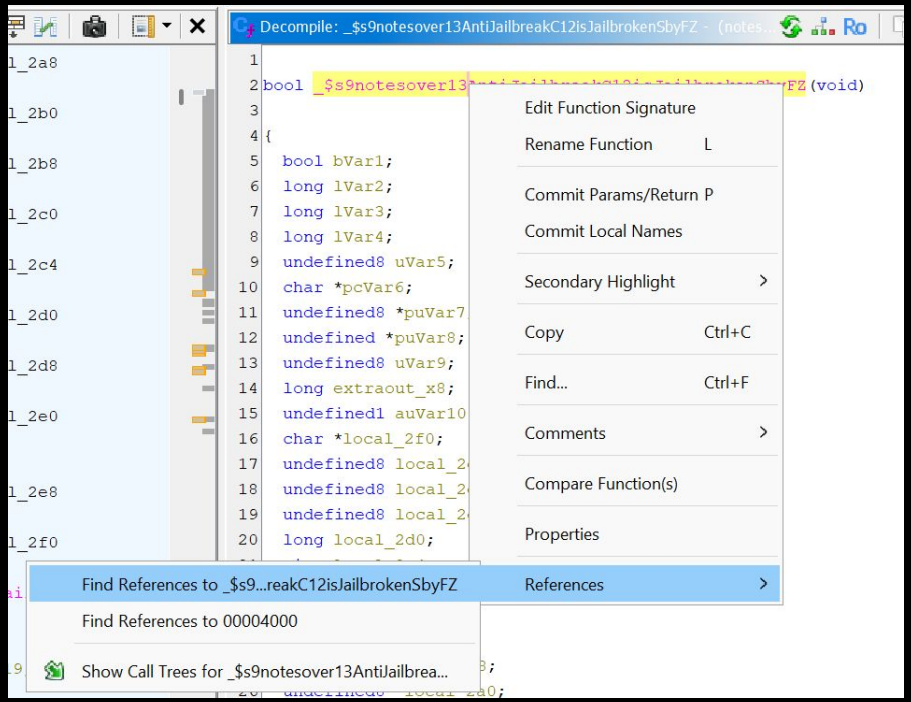




GHIDRA

Challenge 01 - notesover - Ghidra

Então podemos clicar com o botão direito em cima do nome do método e em "References" -> "Find References to ..." para buscar referências a este método. Nosso intuito principal aqui é encontrar o endereço de memória em que está sendo chamado o método isJailbroken(), para tentar contornar essa proteção no momento em que for chamada.

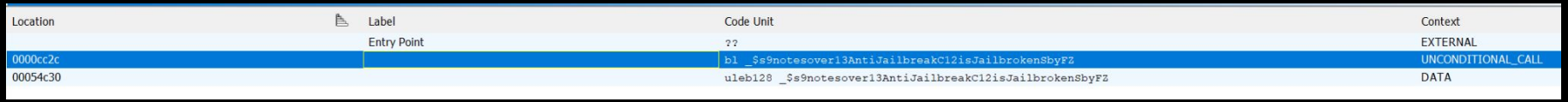


GHIDRA

Challenge 01 - notesover - Ghidra

Então após isso, vemos 3 referências ao método, porém uma em especial chama atenção.

Se lembra de quando falamos da instrução bl ? essa instrução é responsável por chamar um determinado método, neste caso o método isJailbroken(), então vamos acessar o endereço de memória 0000cc2c que é onde existe a instrução bl para o método:

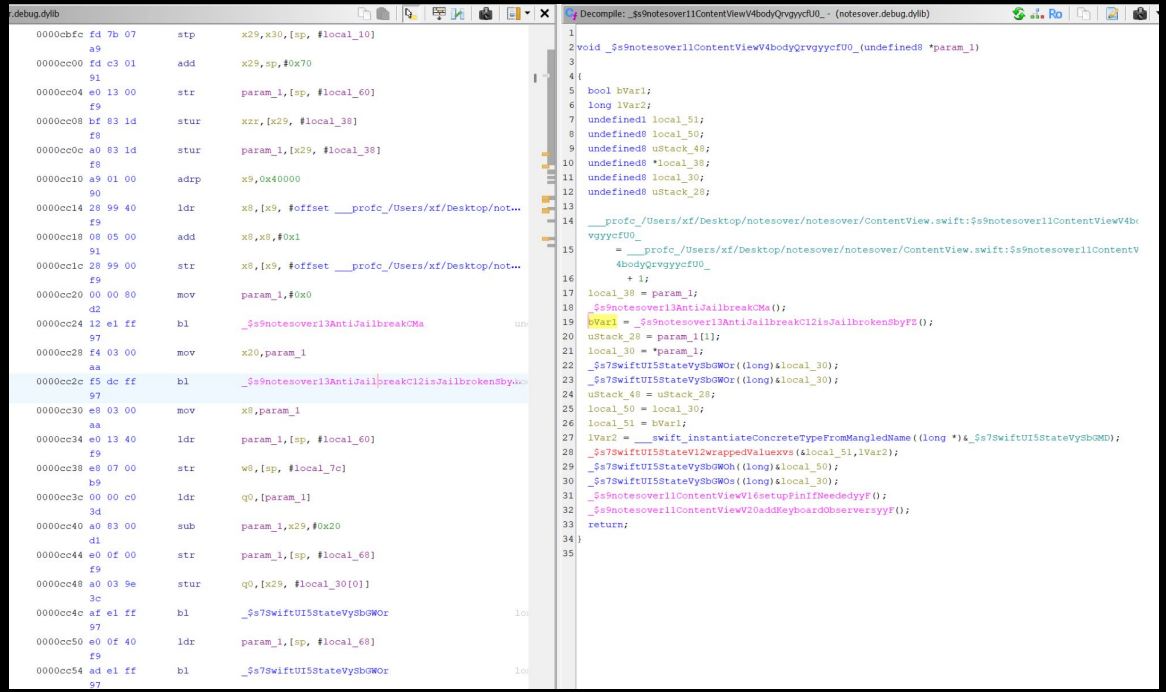


Location	Label	Code Unit	Context
	Entry Point	??	EXTERNAL
0000cc2c		bl _$s9notesover13AntiJailbreakC12isJailbrokenSbyFZ	UNCONDITIONAL_CALL
00054c30		uleb128 _$s9notesover13AntiJailbreakC12isJailbrokenSbyFZ	DATA

GHIDRA

Challenge 01 - notesover - Ghidra

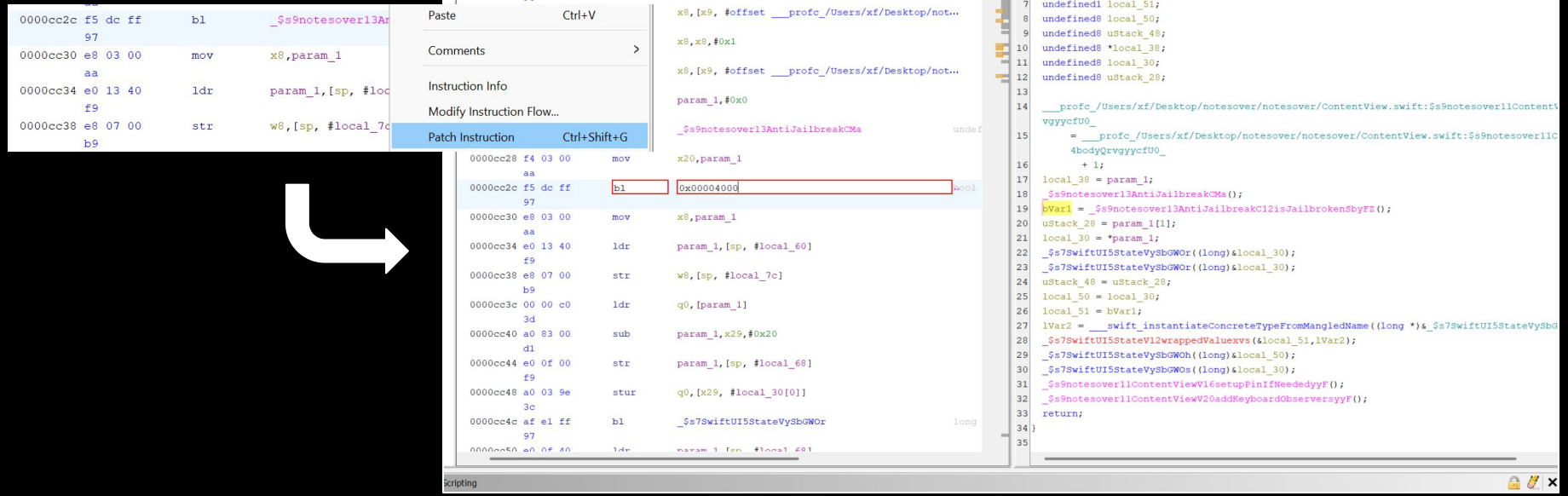
Analisando a referência com a instrução bl para o método isJailbroken(), podemos ver que assim que o aplicativo é iniciado, a *ContentView* chama esse método e armazena seu valor de retorno booleano do mesmo na variável bVar1. Esse valor pode ser true (1), caso o dispositivo esteja com jailbreak, ou false (0), caso não tenha sido detectado *jailbreak* no dispositivo.



GHIDRA

Challenge 01 - notesover - Ghidra

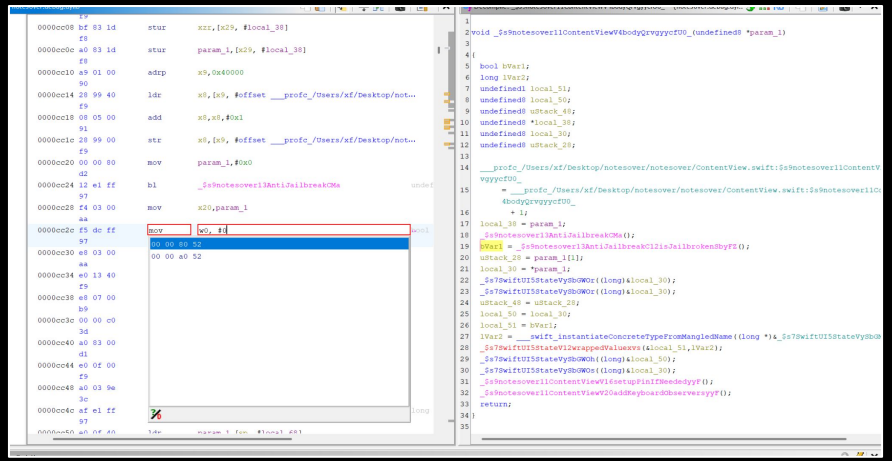
Clicando com o botão direito do mouse na instrução bl para o método isJailbroken(), podemos selecionar a opção "Patch Instruction":

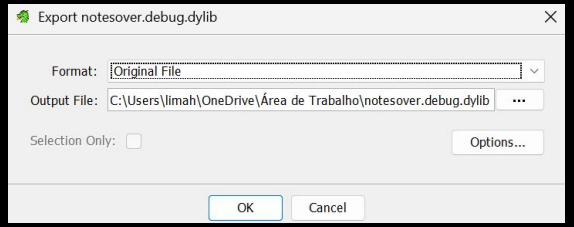


Challenge 01 - notesover - Ghidra

Para forçar um comportamento de "sem jailbreak" e conseguirmos contornar a proteção, devemos modificar a instrução "*bl*" que chama o método *isJailbroken()* para: mov w0, #0, sobrescrevendo a chamada para o método e forçando o valor de retorno para false (0) diretamente no registrador w0.

Como sabemos, em arquiteturas ARM64, o valor de retorno de funções para operações de 32 bits, como booleanos é armazenado nesse registrador, que é responsável por conter os valores de retorno. Essa modificação evita que o método isJailbroken() seja chamado, atribuindo diretamente o valor false ao registrador w0 (w0 = 0). Então, após clicar em "Patch Instruction", modificarmos para: "mov w0, #0" e pressionarmos enter, basta exportamos o nosso binário modificado clicando no menu em File -> *Export Program*, selecione o formato *Original File*.

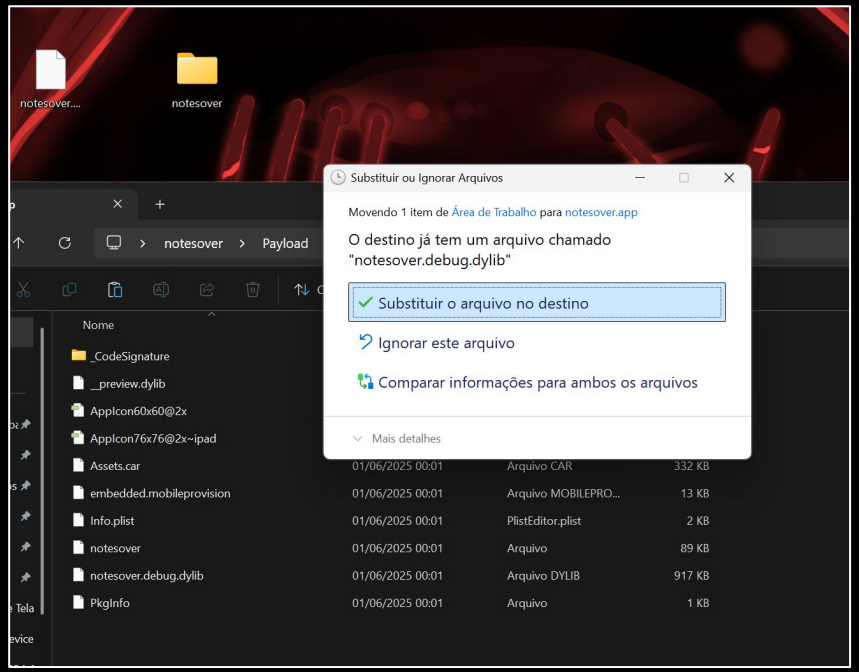


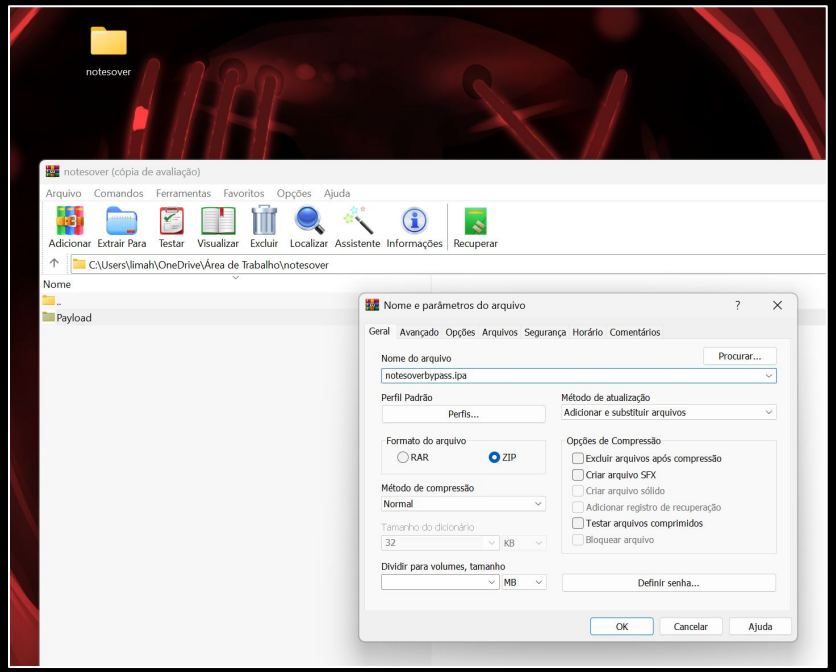


Neste caso, como se trata de uma sobrescrita do método, a instrução NOP também poderia ser utilizada.

Challenge 01 - notesover - Ghidra

Então extraímos o notesover.ipa original, substituímos o notesover.debug.dylib localizado em Payload/notesover.app/, pelo nosso binário de mesmo nome com patch aplicado e exportado do Ghidra. Em seguida, reempacotamos o .ipa alterado em formato zip.

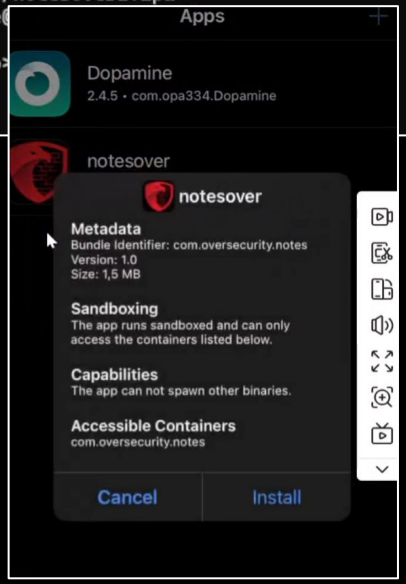




Challenge 01 - notesover - Anti-Jailbreak Bypass com Ghidra

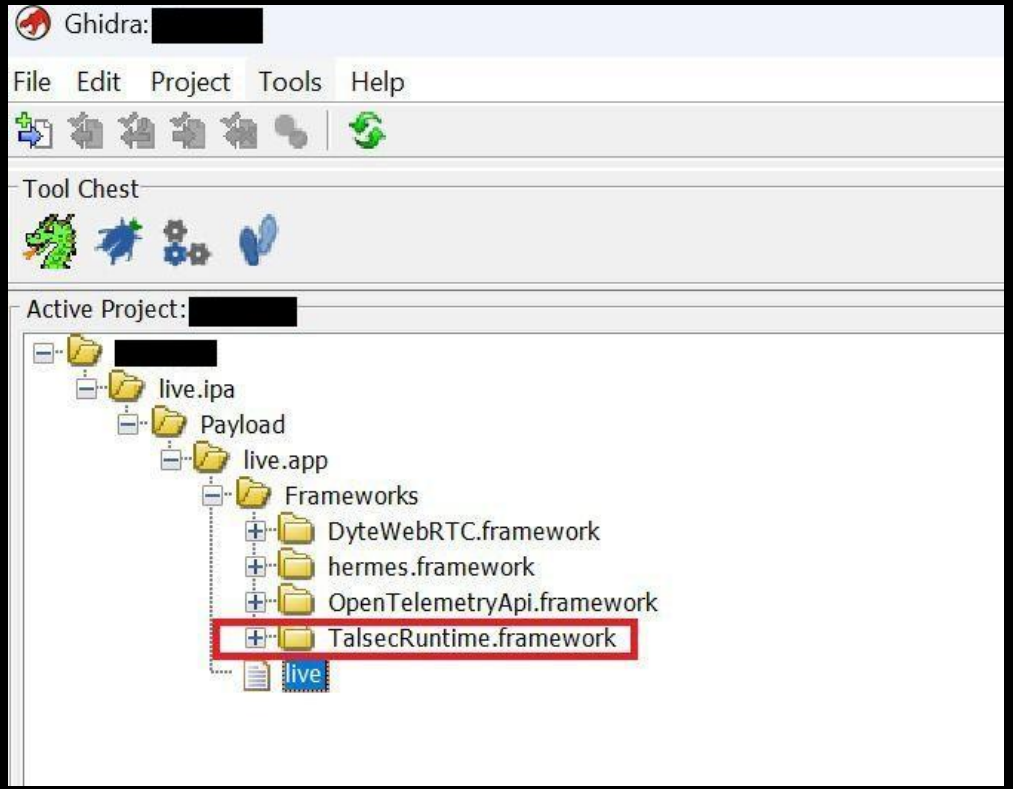
Agora transferimos o nosso .ipa via scp, instalamos no dispositivo e foi possível bypassar o Anti-Jailbreak.

```
PS C:\Users\      \OneDrive\Área de Trabalho> scp .\notesover2.ipa mobile@192.168.31.214:"/private/var/mobile/Library/Mob
ile Documents/com~apple~CloudDocs/Downloads/notesover2.ipa"
(mobile@192.168.31.214) Password for mobile@
notesover2.ipa                                100%  390KB   5.8MB/s   00:00
PS C:\Users\      \OneDrive\Área de Trabalho>
```



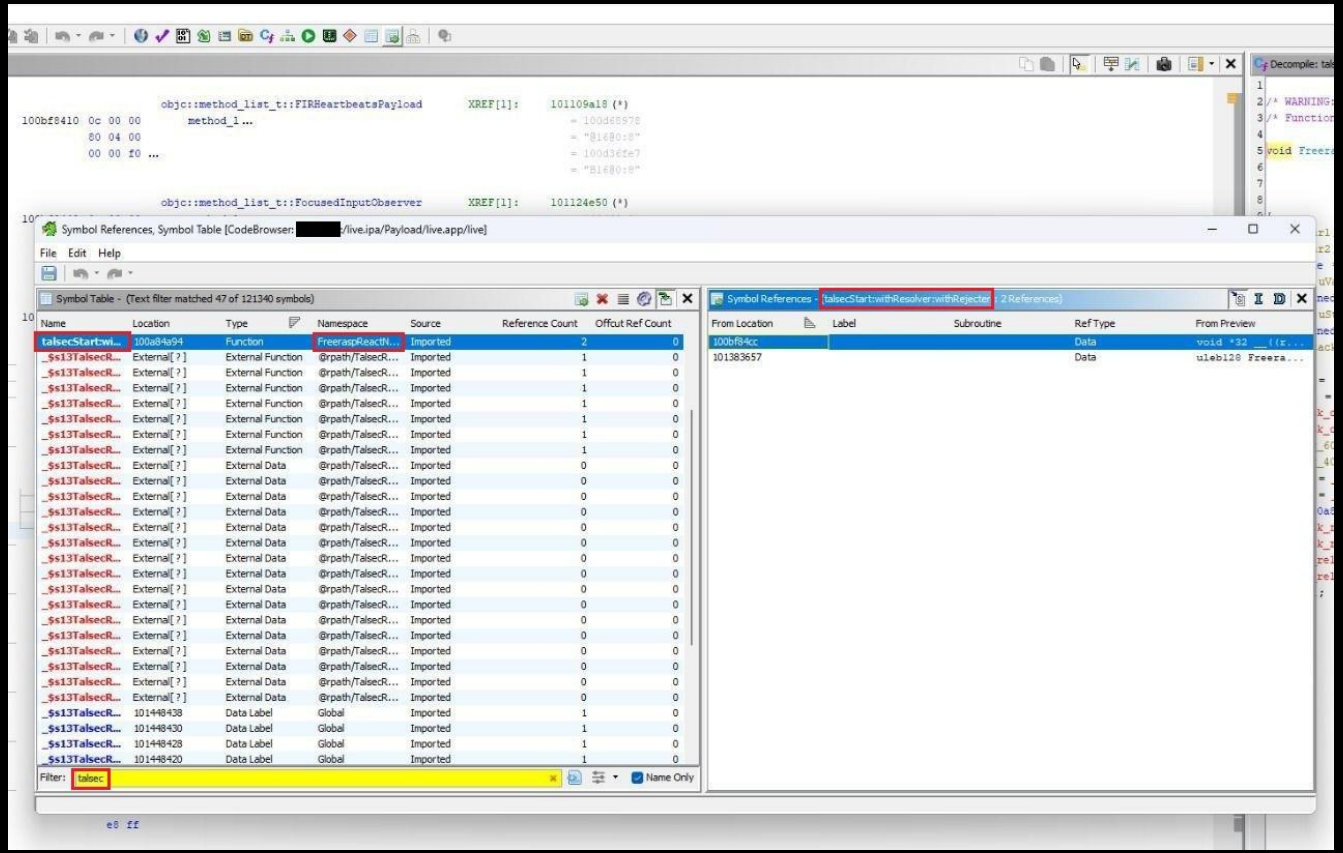


iOS freeRASP React Native Bypass

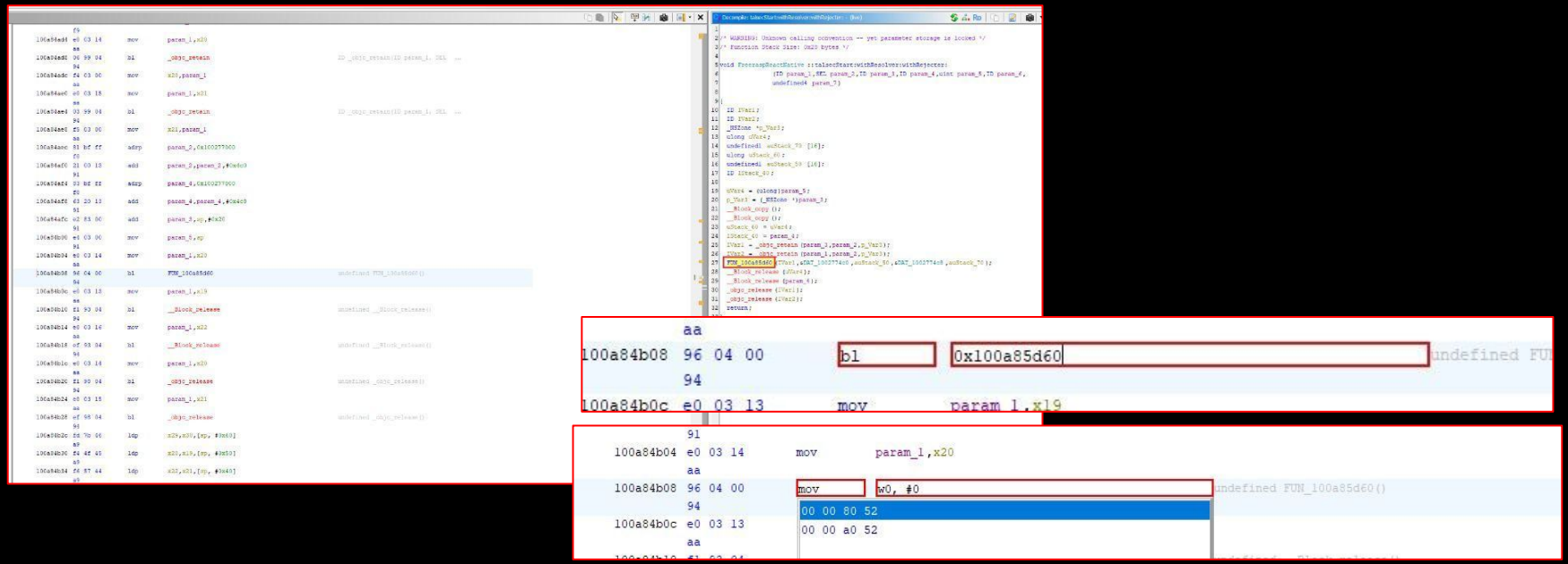


OVER SECURITY INTELLIGENCE

iOS freeRASP React Native Bypass



iOS freeRASP React Native Bypass



Realizando patch no método **talsecStart:withResolver:withRejecter:** do freeRASP e sobrescrevendo-o. Na prática, isso impede que o framework inicialize suas proteções, fazendo com que mecanismos como detecção de jailbreak, hooks e outras features de segurança nunca sejam carregados.

iOS freeRASP React Native Bypass

Project: iOS freeRASP React Native Bypass

Try this code out now by running

```
$ frida --codeshare rodolfomarianocy/ios-freerasp-react-native-bypass -f YOUR_BINARY
```

```
/*
    iOS freeRASP React Native Bypass by rodolfomarianocy
    frida -U -f <bundle_identifier> --codeshare rodolfomarianocy/ios-freerasp-react-native-bypass
    https://github.com/rodolfomarianocy/iOS-freeRASP-React-Native-Bypass
    https://github.com/rodolfomarianocy/Tricks-Pentest-Android-and-iOS-Applications
*/
console.warn("[+] iOS freeRASP React Native Bypass...")
if (ObjC.available) {
    try {
        Interceptor.replace(
            ObjC.classes.FreeraspReactNative['- talsecStart:withResolver:withRejecter:'].implementation,
            new NativeCallback(function() {}, 'void', [])
        );
    } catch (error) {
        console.log(error.message);
    }
} else {
    console.log("[-] ObjC Runtime unavailable");
}
```

Fingerprint: 8a68bd2cd13b22dc15616836ab7977b474b3e2afb332904bf00a81cccf8b5399


SIG // ACTIVE
SCAN // DEEP
RES // 1920x1080
THREAT LEVEL: CRITICAL
0101010101
1001100110
010101010101
0011001110
0101'1010101
101110110
011010101
CONNECTION // ENCRYPTED
SESSION // STEALTH
STATUS // OPERATIONAL
LAT 00.0000
LNG 00.0000
ELEV 00.00
00011001
1001101
OVER
SECURITY INTELLIGENCE
ANALYZE. EXPLOIT. DOMINATE.